

Full length article

A multi-population cooperative coevolution artificial bee colony algorithm for partial multi-robotic disassembly line balancing problem considering preventive maintenance scenarios

Jun Guo, Yang Li, Baigang Du^{*}, Xiang Sun, Kaipu Wang

School of Mechanical and Electronic Engineering, Wuhan University of Technology, Wuhan 430070, China
Hubei Digital Manufacturing Key Laboratory, Wuhan 430070, China

ARTICLE INFO

Keywords:

Multi-robotic workstation
Preventive maintenance
Partial disassembly line balancing
Multi-population cooperative coevolution
Artificial bee colony

ABSTRACT

Existing literature on the robotic disassembly line balancing problem often assume that robots are always in good working condition. But in fact, due to the inevitable aging of the machines, robots may break down unexpectedly. And the original task assignment would become infeasible, which may lead to unplanned shutdowns of the disassembly line. To tackle this situation, a partial multi-robotic disassembly line balancing problem considering preventive maintenance scenarios (PMRDLBP-PM) is proposed. The objectives of the PMRDLBP-PM not only encompass the traditional goals of robotic disassembly line balancing problem, such as cycle time and disassembly profitability, but also take into account the potential additional time and cost incurred from the reconfiguration of workstations necessitated by changes in task allocation. Then, a multi-population cooperative coevolution artificial bee colony (MPCCABC) algorithm is developed. Specifically, to enhance the quality of the initial population and ensure population diversity, the population is divided into four subpopulations, including three high-quality subpopulations generated by heuristic rules based on optimization objectives, and one randomly generated subpopulation. And an adaptive progressive neighborhood search strategy is proposed to improve search efficiency by adjusting the complexity of neighborhood operations based on search feedback. Moreover, a cooperative co-evolution strategy with historical information is adopted to supplement historical optimal information in subpopulation information exchange, increasing computing resource utilization and accelerating convergence speed. Finally, three instances are conducted to test the validity of the proposed model and algorithm. The results demonstrate that the MPCCABC can achieve superior performance for the considered PMRDLBP-PM.

1. Introduction

With the rapid iteration of intelligent products, a large number of end-of-life (EOL) products containing recyclable components are generated annually [1,2]. The disassembly and reuse of those EOL products into their components can reduce the consumption of raw materials and energy [3,4]. The traditional disassembly process is completed manually and continuing to use traditional manual disassembly will consume a lot of human resources due to the number of EOL products being a growing trend [5,6]. In recent years, benefiting from the advancement and improvement of technologies, industrial robots have been widely applied in remanufacturing systems [7]. Robotic disassembly can enhance the productivity of the disassembly line and

alleviate the physical burden on workers [8]. A well-designed robotic disassembly line can significantly enhance production output [9]. Therefore, the robotic disassembly line balancing problem (RDLBP) is one of the important research topics for the robotic disassembly line.

At present, most studies on RDLBP assume that the robot is continuously available. However, due to the inevitable aging of the machines, the robots may break down unexpectedly and need maintenance [10]. Once a robot is under maintenance, the whole disassembly system will shut down and the original task assignment will become infeasible [11]. It is noteworthy that proactive planning of preventive maintenance (PM) scenarios in RDLBP can help to address unforeseeable failures. Furthermore, to ensure production efficiency, RDLBP that considers PM scenarios must not only reassign tasks but also ensure load balancing

^{*} Corresponding author at: School of Mechanical and Electronic Engineering, Wuhan University of Technology, Wuhan 430070, China.
E-mail address: dbg767@163.com (B. Du).

among robots on the disassembly line. Therefore, introducing PM scenarios into the RDLBP is a challenging research problem.

Recently, RDLBP has gradually attracted more and more attention from experts and scholars. Deniz and Ozcelik [12] employed robots to undertake hazardous tasks, aiming to enhance labor safety. Cil et al. [13] assumed that each robot has a different disassembly efficiency in RDLBP and proposed a mathematical model to minimize cycle time. Liu et al. [14] considered the obstacle-avoiding path of the industrial robot's end-effector to avoid the collision problem in RDLBP. To further enhance the efficiency of the disassembly line, Liu et al. [15] then investigated the collaborative optimization problem between RDLBP and disassembly sequence planning. Zhou and Bian [16] considered the tool change time of robots, as well as the varying disassembly skills and efficiencies among different robots, and proposed a sustainable RDLBP to optimize cycle time and total energy consumption. Wu et al. [17] studied the RDLBP with stochastic operation times considering the uncertainty of EOL product states. Xu et al. [18] considered the impact of disassembly failures on the operation of the disassembly line and established a stochastic RDLBP model to optimize both the profit and energy consumption of the disassembly line. Wang et al. [19] considered U-shaped RDLBP to optimize cycle time and devised 10 heuristic rules to attain rational task planning. The above studies on RDLBP consider a simple model, that is, there is only one robot in each workstation. To further improve the disassembly efficiency, some scholars began to study the multi-robot workstation model, which offers several advantages over the single-robot disassembly line, including shorter cycle times and reduced material costs. Fang et al. [20] investigated the multi-robot disassembly line balancing problem (MRDLBP) and developed a corresponding mathematical model intending to improve disassembly line efficiency and reduce energy consumption. Ming et al. [21] formulated a mathematical model for the extended type of MRDLBP, considering the variations in robot efficiency on multi-robot disassembly lines. To raise the resource utilization rate and reduce environmental pollution, Liu et al. [22] proposed an MRDLBP model to optimize the number of robots, hazardous task cost, and CO₂ saving rate. Fang et al. [23] studied a resource-constrained MRDLBP to optimize the number of robots when the total resources of each workstation are limited. Qin et al. [22] investigated U-shaped MRDLBP and developed a mixed integer model to optimize disassembly profits and carbon emissions.

In the above studies, disassembly mode exclusively pertains to complete disassembly, i.e., the recovery of all components from the EOL products. While complete disassembly yields more materials, the increasing complexity of product structures can lead to higher time, labor, and resource costs [24]. In contrast, partial disassembly focuses on specific high-value or hazardous components, recovering only select parts to maximize value recovery and reduce processing costs. Operationally, partial disassembly simplifies processes and enhances efficiency [25]. It achieves this by prioritizing high-demand and hazardous components, thereby reducing complexity and time in the dismantling process while ensuring efficient resource utilization [26]. Environmentally, partial disassembly offers more effective control over impacts by reducing waste and pollutants, particularly when handling high-risk components, significantly lowering environmental risks [27]. Guo et al. [28] integrated the noise generated during the disassembly process into the partial disassembly line balancing problem. Durmaz and Budak [29] considered the energy used in the disassembly process and developed a mathematical model aimed at optimizing the total carbon dioxide emissions. Liang et al. [30] constructed a multi-objective model under the constraints of noise pollution requirements and resource limitations to maximize disassembly profit and minimize energy consumption. Therefore, partial disassembly holds strategic importance in supporting sustainability and resource efficiency. Selectively dismantling parts, especially those with significant impacts on resource recovery and the environment, contributes to promoting the implementation of circular economy models, minimizing waste, and promoting resource reuse to the fullest extent possible [31]. However,

there is a little paper study on the partial robotic disassembly line balancing problem (PRDLBP). Zeng et al. [32] considered the tool replacement of the robot in PRDLBP and established a multi-objective mathematic model that optimized the energy consumption and cost of the disassembly line. Yin et al. [33] established a mathematical model of PRDLBP aiming at the minimization of cycle time, peak energy consumption, total energy consumption, and improved hazard index.

All the above literature assumes that robots can always work properly, which is unrealistic in real production situations [34]. Due to the aging, failure, or maintenance action, the disassembly equipment has to close and become unavailable [35]. Once the equipment is stopped, the original task allocation plan will become infeasible, leading to a cessation of activities across the entire disassembly line [36]. The conventional approach to address this situation is to employ corrective maintenance and rebalancing techniques after shutdown, redistributing relevant tasks to other robots [37]. However, this method results in significant downtime losses and incurs high adjustment costs. In contrast to the reactive nature of corrective maintenance, PM is characterized by its proactive and planned approach, which can be leveraged to mitigate the risk of downtime and disruptions caused by unexpected failures. Based on this, utilizing the determinacy of PM to pre-plan normal working scenarios for all robots under normal working conditions, as well as PM scenarios for certain robots. When a robot is scheduled for maintenance, the corresponding operation allocation plan is adopted directly. However, there is limited research on the disassembly line balancing problem considering preventive maintenance. Guo et al. [38] first addressed preventive maintenance scenarios within the context of the partial disassembly line balancing problem. Based on this, Zeng et al. [39] utilized an actual microwave disassembly line to demonstrate that integrating preventive maintenance into the disassembly line balancing problem can enhance disassembly line efficiency. The above two studies focus on the single-station disassembly model. In the current landscape of rapidly shrinking product life cycles, however, higher efficiency multi-robot workstations and robotic disassembly modes are more aligned with the interests of disassembly enterprises. Therefore, to enhance the operational efficiency of the disassembly line, this paper presents the partial multi-robot disassembly line balancing problem considering preventive maintenance (PMRDLBP-PM).

RDLBP is an NP-hard problem, and PMRDLBP-PM is more intricate because it considers multiple production scenarios. Many metaheuristic algorithms are widely used in DLBP-related problems due to they can generate acceptable solutions with relatively little computational time, such as simulated annealing [40], ant colony optimization [41], fruit fly optimization [42], flower pollination algorithm [43], social engineering optimizer [44], whale optimization algorithm [45], and artificial bee colony (ABC) algorithm [46]. Among them, the ABC algorithm is a promising metaheuristic algorithm in solving DLBP-related problems [47], and has succeeded in showing its competitiveness to other metaheuristic algorithms [48,49]. The strengths of the ABC algorithm lie in its fewer control parameters, simple structure, and strong exploration capability [50]. Despite its advantages, the traditional ABC algorithm often gets trapped in local optima when applied to large or medium-sized problems. It should be noted that the multi-population division is useful to maintain population diversity and avoid premature convergence [51]. Therefore, this paper proposes a multi-population cooperative coevolution artificial bee colony algorithm (MPCCABC) for solving the PMRDLBP-PM. Specifically, the population is divided into four subpopulations according to the corresponding heuristic rules that are designed based on the characteristics of the problem to increase population diversity; an adaptive progressive neighborhood search (APNS) strategy is presented to promote the efficiency of exploration; a cooperative co-evolution strategy with historical information (CCHI) is introduced that uses the current best search solutions and historical optimization solutions of all four subpopulations when the solution is stagnant. By CCHI strategy, four subpopulations can exchange their best search experiences and increase algorithm efficiency. The main

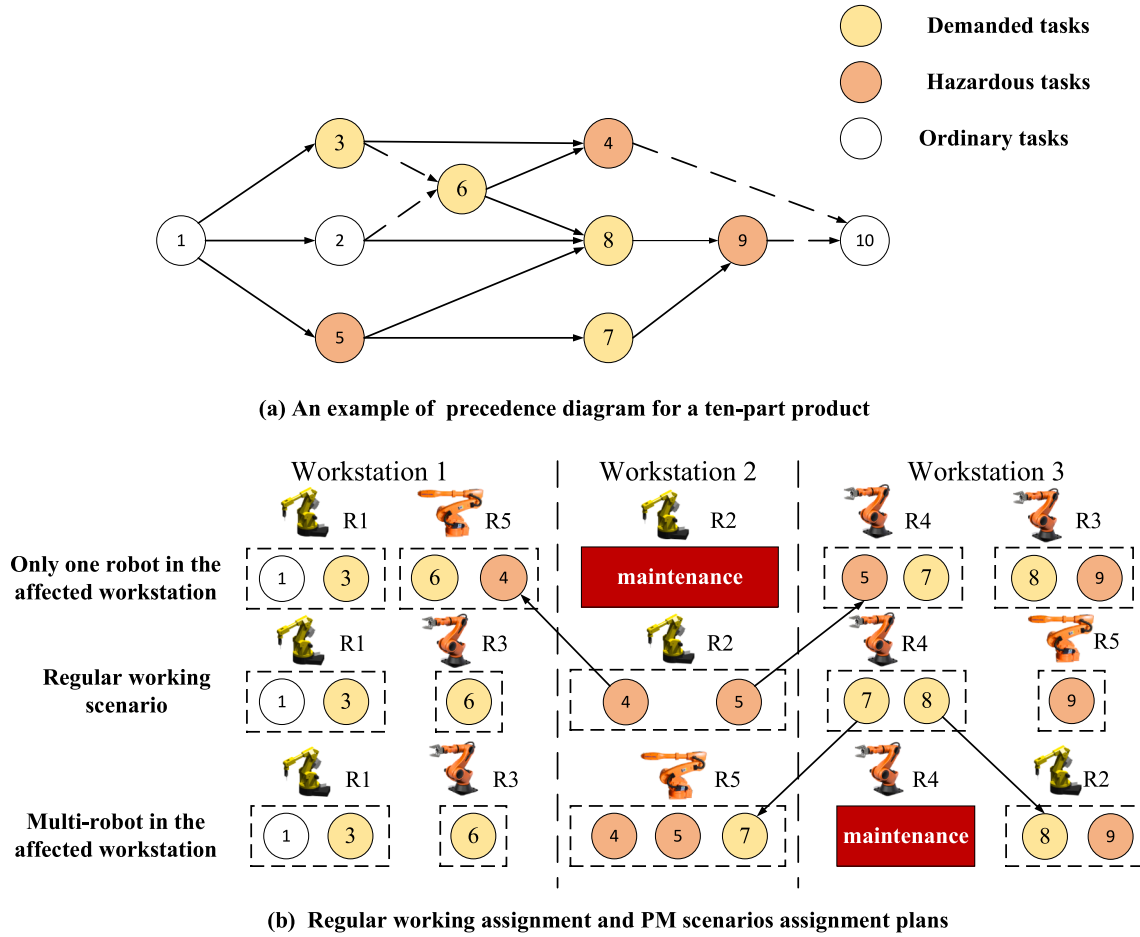


Fig. 1. The precedence graph of an EOL product and scheduling changes between different scenarios.

contributions of this article include three aspects:

- (1) To the best of our knowledge, this paper is the first attempt to introduce PM scenarios into the partial multi-robotic disassembly line problem. And a mathematical model is proposed to optimize cycle time, profit of disassembly, the energy consumption of robots, and the cost of task reassignment simultaneously.
- (2) A novel MPCCABC algorithm is developed according to the characteristics of PMRDLBP-PM. In MPCCABC, to enhance population diversity, three specially designed heuristic rules are proposed to generate three high-quality subpopulations. And an APNS strategy is presented to accelerate the exploration efficiency within the subpopulations. Finally, a CCHI strategy is adopted in the subpopulation information exchange phase to increase computing resource utilization.
- (3) By performing comparing experiments on multi-group cases, the work validates the superiority and efficiency of the proposed algorithm in solving the PMRDLBP-PM.

The remainder of this article is organized as follows. Section 2 formulates the mathematical description of PMRDLBP-PM. Section 3 proposes the MPCCABC algorithm. Section 4 analyzes the experimental results. Section 5 reports the conclusion of the study and gives some directions for future work.

2. Problem description and modeling

2.1. Problem description

PMRDLBP-PM comprises three processes: the first step involves allocating n disassembly tasks to m workstations, the second step involves allocating w robots to m workstations, and the third step involves assigning tasks in each workstation to the corresponding robots. PMRDLBP-PM considers two kinds of production scenarios: the regular working scenario and PM scenarios. During the regular working scenario, all robots are available. However, in the PM scenario, a certain robot is under maintenance, and the original task allocation plan becomes infeasible. Hence, the task allocation has to be readjusted to maintain the continuity of the production system. In this paper, we try to give the scheduling plan of this PM scenario in advance, so that when the certain robot needs maintenance, the disassembly line can switch from the regular working scenario to the PM scenario quickly to maintain production. Obviously, minimizing the number of tasks that require adjustment results in a shorter downtime for the disassembly line. Therefore, in addition to the traditional objectives of minimizing cycle time, energy consumption, and maximizing profit, PMRDLBP-PM also considers the cost of task readjustment.

Take a ten-part EOL product as an example to illustrate the reassignment of disassembly tasks when a certain robot is under PM. Fig. 1 (a) depicts the precedence relationships among the tasks of the EOL product using an AND/OR diagram, where the straight line and dotted line represent the AND and OR relationships, respectively. An AND precedence specifies that a task can only commence once all preceding tasks have been disassembled, reflecting a sequential dependency.

Conversely, an OR precedence indicates that the initiation of a task requires the completion of at least one (but not necessarily all) of the preceding tasks, allowing for a degree of flexibility and choice in the order of task execution. As shown in Fig. 1(a), tasks 2, 5, and 6 are AND predecessors of task 8. This means task 8 can only be removed after tasks 2, 5, and 6 have been completed. Furthermore, the relationship between tasks 2 and 3 with task 6 reveals an OR precedence. This means task 6 can be removed once either task 2 or task 3 has been disassembled. Fig. 1(b) shows a regular working scenario and two PM scenarios in which robot 2 and robot 4 are under maintenance. In the regular working scenario, tasks can be assigned to every robot. When robot 2 is under maintenance, tasks 4 and 5 have to be reassigned to robot 5 and robot 4, and the assignment of robots also needs to be changed accordingly to regenerate a feasible task assignment scheme. Similarly, when robot 4 is under maintenance, task 7, task 8 and robots have to be reassigned.

The assumptions of the RPDLBP-PM are as below:

- (1) The hazardous and demanded tasks are given in advance and must be removed, while other parts are not necessary;
- (2) The disassembly time of tasks is deterministic;
- (3) The precedence relationships among tasks are depicted by AND/OR precedence;
- (4) Each workstation should be assigned to at least one robot;
- (5) Each robot can disassemble any task, but different robots have different disassembly efficiency;
- (6) In each PM scenario, only one robot is under maintenance.

3. Notations

Indices and sets	
i	The index of tasks, $i = \{1, 2, 3, \dots, n\}$
k	The index of workstations, $k = \{1, 2, 3, \dots, m\}$
r	The index of robots, $r = \{1, 2, 3, \dots, w\}$
c	The index of scenarios, $c = \{0, 1, 2, \dots, s\}$. ($c = 0$ represents the regular working scenario; others represent PM scenarios)
PAND(i)	Set of the AND immediately preceding tasks of task i .
POR(i)	Set of the OR immediately preceding tasks of task i .

Parameters

n	Number of disassembly tasks.
m	Number of workstations.
w	Number of robots.
s	Number of scenarios.
m_w	Number of robots assigned to each workstation
m_{wmax}	Maximum number of robots assigned to each workstation
h_i	If task i is hazardous, $h_i = 1$, else, $h_i = 0$
d_i	If task i is demanded, $d_i = 1$, else, $d_i = 0$.
t_{ir}	Consumed time of performing task i in robot r .
t_{or}	Replacement time of the tool in robot r .
re_i	Revenue of task i .
c_r	Unit time cost of the robot.
we_r	Unit time working energy cost of robot r .
se_r	Unit time standby energy cost of robot r .
ce_r	Unit time replacement energy cost of robot r .
$C_{r,c}$	Completion time of robot r on scenario c .
CT_c	Cycle time of the disassembly line on scenario c .

Decision variables

$x_{irk,c}$	If task i is allocated to robot r at workstation k on scenario c , $x_{irk,c} = 1$, else, $x_{irk,c} = 0$.
$y_{rk,c}$	If robot r is allocated to workstation k on scenario c , $y_{rk,c} = 1$, else, $y_{rk,c} = 0$.
z_{iir}	If task i and task i' need different tools in robot r , $z_{iir} = 1$, else, $z_{iir} = 0$.
u_{iir}	If task i is the immediate predecessor of task i' in robot r , $u_{iir} = 1$, else, $u_{iir} = 0$.
$M_{r,c}$	If robot r is under maintenance on scenario c , $M_{r,c} = 1$, else, $M_{r,c} = 0$.

3.1. Objectives and constraints

For any scenario c ($0 \leq c \leq s$), the mathematical model of MPR-PDLBP is constructed as follows:

4. Objective functions

1. Minimize cycle time

$$\min f_1 = CT_c \quad (1)$$

2. Maximize the disassembly profit

$$\max f_2 = \sum_{i=1}^n \sum_{k=1}^m \sum_{r=1}^w re_i \cdot x_{irk,c} - \sum_{k=1}^m \sum_{r=1}^w C_{r,c} \cdot y_{rk,c} \quad (2)$$

3. Minimize the total energy consumption of robots

$$\min f_3 = \sum_{r=1}^w (we_r \cdot \sum_{i=1}^n \sum_{k=1}^m t_{ir} \cdot x_{irk,c}) + \sum_{r=1}^w (se_r \cdot (CT_c - C_{r,c})) + \sum_{r=1}^w (ce_r \cdot \sum_{i=1}^n \sum_{i' \neq i} \sum_{k=1}^m x_{irk,c} \cdot x_{i'rk,c} \cdot z_{iir} \cdot u_{iir} \cdot t_{ir}), \quad (3)$$

4. Minimize total alteration of tasks and robots

$$\min f_4 = \sum_{i=1}^n \sum_{k=1}^m \sum_{r=1}^w |(x_{irk,0} - x_{irk,c})| + |(y_{rk,0} - y_{rk,c})| \quad (4)$$

5. Constraints

$$\sum_{k=1}^m \sum_{r=1}^w x_{irk,c} \leq 1, \forall i \in \{1, \dots, n\} \quad (5)$$

$$\sum_{k=1}^m y_{rk,c} = 1, \forall r \in \{r | M_{r,c} = 0\} \quad (6)$$

$$1 \leq \sum_{r=1}^w y_{rk,c} \leq m_{wmax}, \forall k \in \{1, \dots, m\} \quad (7)$$

$$\sum_{k=1}^m \sum_{r=1}^w x_{irk,c} \geq h_i, \forall i \in \{1, \dots, n\} \quad (8)$$

$$\sum_{k=1}^m \sum_{r=1}^w x_{irk,c} \geq d_i, \forall i \in \{1, \dots, n\} \quad (9)$$

$$x_{irk,c} \leq \sum_{r'=1}^w \sum_{k'=1}^m x_{gr'k',c}, \forall i \in \{1, \dots, n\}, k \in \{1, \dots, m\}, r \in \{1, \dots, w\}, g \in \text{PAND}(i) \quad (10)$$

$$x_{irk,c} \leq \sum_{r'=1}^w \sum_{k'=1}^m \sum_{g \in \text{POR}(i)} x_{gr'k',c}, \forall i \in \{1, \dots, n\}, k \in \{1, \dots, m\}, r \in \{1, \dots, w\} \quad (11)$$

$$\sum_{i=1}^n \sum_{k=1}^m x_{irk,c} \cdot y_{rk,c} = 0, \forall r \in \{r | M_{r,c} = 1\} \quad (12)$$

$$C_{r,c} = \sum_{i=1}^n \sum_{k=1}^m (x_{irk,c} \cdot t_{ir} + \sum_{i' \neq i} x_{irk,c} \cdot x_{i'rk,c} \cdot z_{iir} \cdot u_{iir} \cdot t_{or}), \forall r \in \{1, \dots, w\} \quad (13)$$

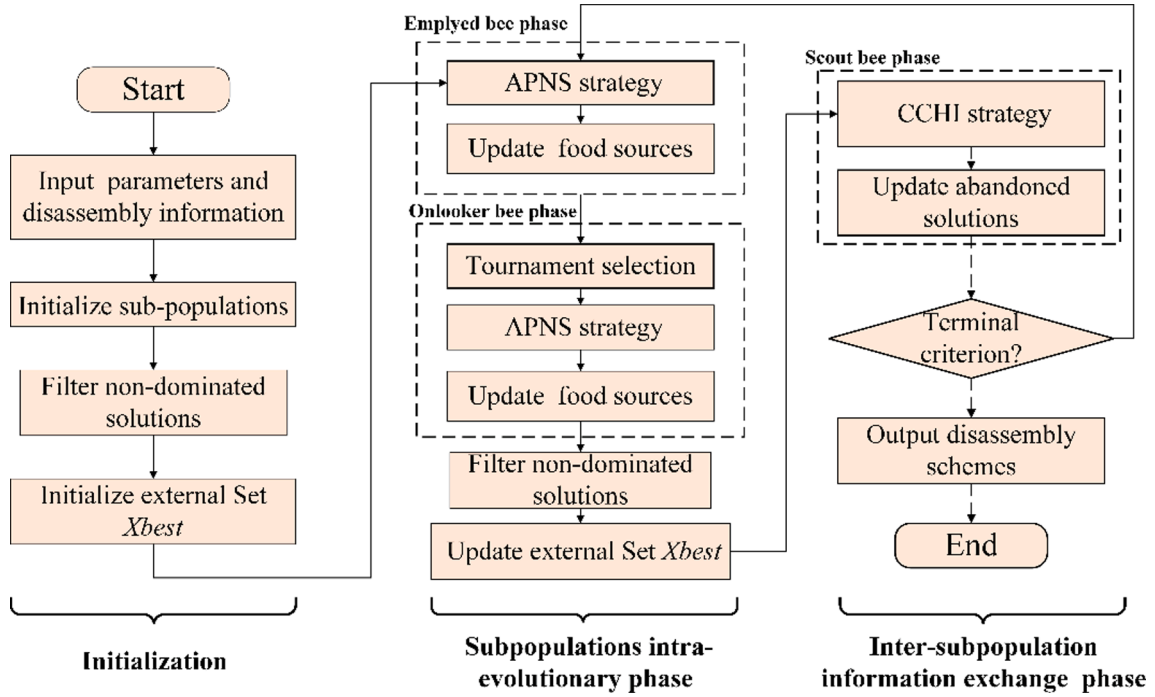


Fig. 2. Flow chart of MPCCABC.

$$CT_c = \max_{k=1}^n \left\{ \sum_{r=1}^w \sum_{i=1}^n C_{r,c} \cdot y_{rk,c} \right\} \quad (14)$$

Eq. (1) minimizes the cycle time of the disassembly line. Eq. (2) maximizes the disassembly profit of the EOL product which consists of the total revenue of selected disassembly parts and the cost of running the robots. Eq. (3) minimizes the total energy consumption of the robots used in the disassembly line which consists of the energy cost of robots in the working situation, the energy cost of robots while on standby, and the tool replacement energy cost of the robots, respectively. Eq. (4) minimizes the cost of reassignment from the regular working scenario to PM scenarios which consists of the cost of task reassignment and robot reassignment.

Eq. (5) represents that not all parts will be disassembled; however, any part that is disassembled can only be assigned to one robot at one workstation. Eq. (6) specifies that all robots not under maintenance will be assigned to a workstation, and each robot can only be assigned to one workstation. Eq. (7) indicates that the number of robots assigned to each workstation should be between 1 and m_{wmax} . Eq. (8) and Eq. (9) indicate that the hazardous and demanded tasks should be removed. The constraints given in Eq. (10) and Eq. (11) are related to the precedence relationship among tasks. Eq. (10) governs the task i can only be removed after all of its AND predecessors (i.e. $\forall g \in PAND(i)$) have been disassembled. Eq. (11) guarantees that task i cannot be removed unless one of its OR predecessors (i.e. $\exists g \in POR(i)$) has been disassembled. Eq. (12) suggests that if one robot is under maintenance and won't be assigned any task. Eq. (13) expresses the working time of robot r , encompassing both the disassembly time and the tool replacement time of the robot. Eq. (14) shows that CT equals the maximum finish time of any station.

5.1. Pareto optimality

In this paper, Pareto optimality is used to compare and select solutions for the four-objective mathematical model [52]. Pareto optimality is a fundamental concept in multi-objective optimization, describing a situation where no objective can be improved without worsening at least one other objective. A solution is considered Pareto optimal if there is no

other solution that dominates it in terms of all objectives.

Pareto dominance is a principle used to compare two solutions in the context of multiple objectives. Given two solutions, A and B , solution A is said to dominate solution B if and only if A is at least as good as B in all objectives, and A is strictly better than B in at least one objective. A solution is termed non-dominated if it is not dominated by any other solution in the solution space. The set of all non-dominated solutions is known as the Pareto front. Solutions on the Pareto front represent the best trade-offs among the objectives, allowing decision-makers to choose from these solutions based on their preferences and priorities.

6. The proposed MPCCABC algorithm

The ABC is a popular metaheuristic algorithm for addressing DLBP-related problems. However, it faces challenges in convergence speed and exploitation capabilities when solving complex problems. As an extension of DLBP, the introduced PMRDLBP-PM is more complex. Therefore, to enhance algorithmic performance, we propose a novel algorithm based on multi-population and the reuse of elite genes. In the proposed method, three modifications are introduced to accelerate convergence and prevent entrapment in local optima. Firstly, the initialization method of solutions significantly impacts the quality of the final solution. Thus, to improve population quality and maintain diversity, the population is divided into four subpopulations based on the optimization objectives, with each subpopulation employing different initialization rules. Secondly, to avoid local optima, an adaptive progressive neighborhood search strategy is proposed, selecting the neighborhood structure with appropriate operational complexity based on search outcomes. A neighborhood structure with lower complexity is chosen when results are satisfactory, whereas a neighborhood structure with higher complexity is selected when improvements are not observed over multiple iterations. Finally, to conserve computational resources and enhance efficiency, a cooperative co-evolution strategy with historical information is introduced, utilizing the non-dominated solution from the previous iteration to generate new food sources. The flow chart of MPCCABC is shown in Fig. 2, and the pseudo-code of the MPCCABC is

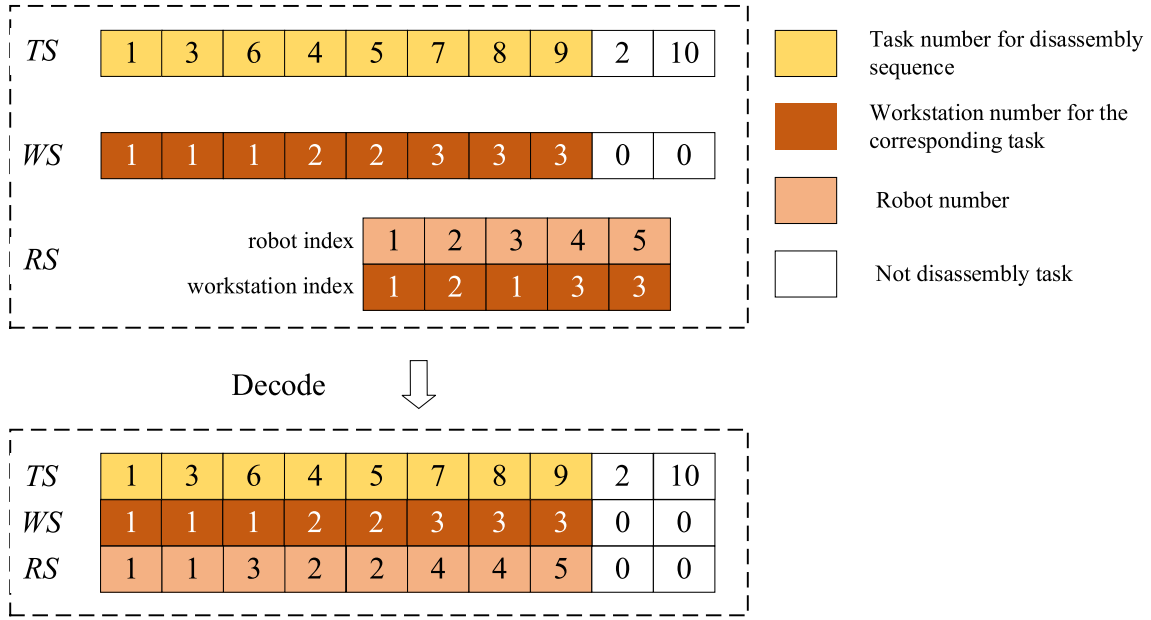


Fig. 3. Three-dimensions encoding and decoding method.

presented in Algorithm 1.

Algorithm 1: MPCCABC

Input: population size np , the ratio between subpopulations θ , termination iteration $maxTime$, maximum trial number on a solution $uplimit$, and the number of scenarios c .

Output: Pareto solution X_{best}

- 1 Initialize four subpopulations
- 2 **for** $sn = 0$ to c **do**
- 3 **while** $g < maxTime$ **do**
- 4 **Information interaction within subpopulations:**
- 5 **for** each solution in every subpopulation **do**
- 6 Employed bee phase: adopt APNS strategy
- 7 Calculate probability
- 8 Onlooker bee phase: adopt APNS strategy
- 9 **Information interaction between subpopulations:**
- 10 Scout bee phase: update the sources with the CCHI strategy
- 11 Select the learning individual and the being learned individual then
- 12 cross
- 13 Implant characteristics of the being learned individual into the learning individual
- 14 **end for**
- 15 Update the external archives and X_{best}
- 16 $g = g + 1$.
- 17 **end while**
- 18 **end for**
- 19 **Return** X_{best}

6.1. Encoding and decoding

In this study, a three-dimensions encoding and decoding method is adopted to represent the solution which consists of task sequence (TS), robot sequence (RS), and workstation sequence (WS), as shown in Fig. 3. In the encoding process, the sequence of tasks is first determined. The random initialization coding process is as follows: First, randomly select a task i according to the precedence relationship matrix of tasks; Second, update the priority matrix to eliminate the constraint of task i on the remaining tasks; Third, repeat Step 1 and 2 until all hazardous and demanded the tasks have been removed.

There are two stages exist in the decoding process: The first stage is to assign the tasks of the TS to the workstation, and the second stage is to assign those tasks to the corresponding robots at the workstation sequentially. The pseudocode of the decoding process is shown as

follows.

Algorithm 2: Disassemble decoding

Input: task sequence TS, task attribute matrix HD , number of tasks in task sequence n_{ts} , number of workstations m , and the number of robots w .

Output: a solution x

- 1 Calculate the ideal cycle time CT_s according to the total processing time T_{sum} of the tasks in TS, $CT_s = T_{sum}/m$.
- 2 $a = 0$ // when the cycle time constraint is satisfied $a = 1$.
- 3 **while** $a = 1$ **do**
- 4 **for** $i = 1$ to n_{ts} **do**
- 5 Workstation index $k = 1$.
- 6 $WS(i) = k$ //assign the task i to workstation k .
- 7 Calculate the total processing time T_s of the tasks in workstation k .
- 8 **if** $T_s > CT_s$ & $k < m$ **do** // the workstation k processing time exceeds the ideal cycle time
- 9 $k = k + 1$. //open a new station
- 10 **else**
- 11 **if** $T_s > CT_s$ & $k = m$ **do**
- 12 $CT_s = CT_s + 1$.
- 13 **break** // restart the allocation process
- 14 **else**
- 15 **if** $T_s \leq CT_s$ & $k = m$ & $i = n_{ts}$ **do**
- 16 $a = 1$. //end the allocation process
- 17 **end if**
- 18 **end if**
- 19 **end if**
- 20 **end for**
- 21 **end while**
- 22 assign the robots to workstations randomly
- 23 **for** $k = 1$ to m **do**
- 24 randomly assign tasks that belong to workstation k to the robot in workstation k , and then get RS
- 25 **end for**
- 26 Merge the TS, WS, and RS to obtain the set solution x .
- 27 **Return** solution x

6.2. Subpopulation initialization with heuristic rules

The research shows that dividing one population into multiple populations can increase the population diversity and avoid premature convergence, meantime the general initialization method – random generation can quickly generate the initial solutions, but it is difficult to ensure their quality. Designing the corresponding initialization rules according to the characteristics of the optimization target can accelerate

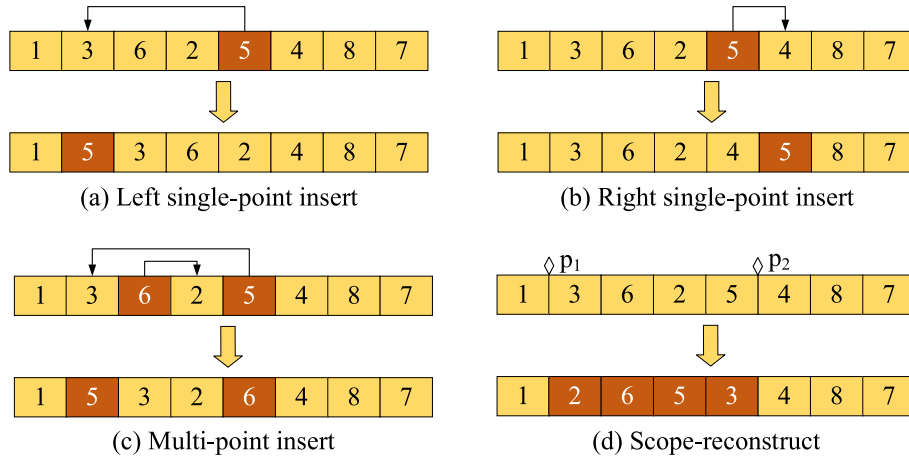


Fig. 4. The neighborhood structures.

the search process of the algorithm [53]. Thus, we divided the population into random initialization population and heuristic initialization population and then fairly divided the heuristic initialization population into three sub-populations according to the characteristics of RPDLBP-PM.

The initialization process of the random subpopulation (subpopulation 1) is shown in Section 3.1. To minimize the CT, subpopulation 2 only removes the tasks that are on the disassembly path of the hazardous and demanding tasks, the procedures are shown in Algorithm 3. To maximize the profit, all the revenue greater than zero tasks will be disassembled in subpopulation 3. To minimize energy consumption, calculate the power consumption of each robot to disassemble parts in the current station when assigning robots to stations, and priority to the low-energy robot in subpopulation 4.

Algorithm 3: Heuristic initialization process for Subpopulations.

Input: population size np , precedence matrix P , the attribute matrix HD , number of tasks n , and the ratio of the number of individuals between subpopulations θ .

Output: Subpopulations 2–4.

```

1  Calculate the number of individuals in each heuristic initialization
   subpopulation ( $N_s$ ) according to Eq. (15) – Eq. (16).
2  for  $i = 1$  to  $3N_s$  do
3    if  $i < N_s$  do
4       $a = 1$ .
5    else
6      if  $i < 2N_s$  do
7         $a = 2$ .
8      else
9         $a = 3$ .
10     end if
11   end if
12   switch the value of  $a$  do // Determine the heuristic initialization rule
13     case 1 // Subpopulation 2
14       Find the parts with the attribute hazardous and demand and enter them
       into the initial processing set ( $Is$ ).
15       while  $Is$  is not empty do
16         Identify the tasks that possess immediate precedence relationships with
         task  $i$  and subsequently include them in the priority set ( $Ps$ ). Following this, task
          $i$  is removed from  $Is$ .
17       end while
18        $d =$  number of set  $Ps$ .
19       Update  $P$ 
20     case 2 // Subpopulation 3
21       for  $i = 0$  to  $n$  do
22         if  $HD(i) = 1$  ||  $re_i > 0$  do
23            $d = d + 1$ .
24         end if
25       end for
26     case 3 // Subpopulation 4
27       for  $i = 0$  to  $n$  do
28         if  $HD(i) = 1$  do
29            $d = d + 1$ .

```

(continued on next column)

(continued)

```

30   end if
31   end for
32   end Switch
33   while  $j < d$  do
34     Obtain the set of available tasks (AS) based on  $P$  then randomly select task  $i$ 
       from AS.
35     Add task  $i$  into  $TS$  then update  $P$ .
36      $j = j + 1$ 
37   end while
38   assign the tasks to each workstation then get  $WS$ .
39   if  $a = 3$  do
40     priority to the low-energy robot when assigning tasks to each robot then get
        $RS$ .
41   else
42     assign the tasks to each robot randomly then get  $RS$ .
43   end if
44   Merge the  $TS$ ,  $WS$ , and  $RS$  to obtain the set solution  $x$ .
45   end for
46   Return Subpopulations 2–4

```

The number of individuals in each subpopulation is calculated as follows:

$$\frac{N_h}{N_r} = \theta \quad (15)$$

$$N_s = \frac{np \cdot \theta}{3 \cdot (1 + \theta)} \quad (16)$$

where N_h is the number of individuals in three heuristic initialization subpopulations. Each heuristic initialization subpopulation comprises N_s individuals, and N_s is one-third of N_h . N_r is the number of individuals in the random initialization subpopulation. The total number of individuals across all four subpopulations is np , which is the sum of N_h and N_r . θ is the ratio between N_h and N_r .

6.3. Adaptive progressive neighborhood search strategy

The subpopulation intra-evolutionary phase is dominated by employed and onlooker bees, and an APNS strategy is proposed to improve search efficiency and prevent employed bees and onlooker bees from falling into local optima. The core idea of this approach involves the selection of an optimal neighborhood structure of appropriate operational complexity based on the detection outcomes. Specifically, if a newly obtained solution represents an enhancement compared to its predecessor, the current improvement direction is promising, and the neighborhood structure of lesser operational complexity is selected for further improvement, facilitating finer adjustments and refinements. Alternatively, if repeated attempts using a lower complexity

neighborhood structure fail to yield improvement, a neighborhood structure with greater operational complexity is chosen to escape local optima and enable deeper exploratory. In this paper, the complexity of a neighborhood structure is determined by the number of tasks it operates on. More tasks result in longer and more complex structures, thereby increasing the depth of the search. Conversely, fewer tasks lead to shorter and simpler structures, resulting in a reduced search depth. The algorithmic outline for the Progressive Neighborhood Search is delineated in Algorithm 4.

The APNS strategy employs four variations of neighborhood structures differing in operational complexity to facilitate subpopulation information transmission. These neighborhood structures are characterized by varying degrees of intricacy, ranked in order of increasing complexity as follows: left single-point insert neighborhood structure, right single-point insert neighborhood structure, multi-point insert neighborhood structure, and scope-reconstruct neighborhood structure. The schematic diagram of neighborhood structures is shown in Fig. 4.

€.

Algorithm 4: The adaptive progressive neighborhood search

Input: solution x and its fitness value $f(x)$
Output: a new solution x_1 generated by APNS

```

1  flag = 1, trial(x) = 0
2  for i = 0 to uplimit do
3    Switch flag do // Determine the type of neighborhood structure
4      case 1
5        Generate a new solution  $x_1$  with the scope-reconstruct neighborhood structure.
6        if  $f(x_1)$  Pareto dominates  $f(x)$  do
7           $f(x) = f(x_1)$ ,  $x = x_1$ , trial( $x_1$ ) = 0, flag = flag + 1.
8        else
9          trial( $x_1$ ) = trial( $x_1$ ) + 1.
10       end if
11     case 2
12       Generate a new solution  $x_1$  with the multi-point insert neighborhood structure.
13       if  $f(x_1)$  Pareto dominates  $f(x)$  do
14          $f(x) = f(x_1)$ ,  $x = x_1$ , trial( $x_1$ ) = 0, flag = flag + 1.
15       else
16         trial( $x_1$ ) = trial( $x_1$ ) + 1.
17         if trial( $x_1$ ) > uplimit / 4 do //
18           flag = flag - 1.
19         end if
20       end if
21     case 3
22       Generate a new solution  $x_1$  with the left single-point insert neighborhood structure.
23       if  $f(x_1)$  Pareto dominates  $f(x)$  do
24          $f(x) = f(x_1)$ ,  $x = x_1$ , trial( $x_1$ ) = 0, flag = flag + 1.
25       else
26         trial( $x_1$ ) = trial( $x_1$ ) + 1.
27         if trial( $x_1$ ) > uplimit / 4 do
28           flag = flag - 1.
29         end if
30       end if
31     case 4
32       Generate a new solution  $x_1$  with the right single-point insert neighborhood structure.
33       if  $f(x_1)$  Pareto dominates  $f(x)$  do
34          $f(x) = f(x_1)$ ,  $x = x_1$ , trial( $x_1$ ) = 0;
35       else
36         if trial( $x_1$ ) > uplimit / 4 do //
37           flag = flag - 1.
38         end if
39       end if
40     end switch
41   end for
42   Return  $x_2$ 

```

Left single-point insert neighborhood structure: Only change the position of one task, which is the neighborhood structure with the least operational complexity. Randomly select a position of the food source, then get the preceding task and the following task of the task at this

position according to the P . This task can be inserted into any position between the preceding task and the following task, while the new position should be on the left of the original position. Since the new position also respects the precedence relations with its preceding task and following task, the newly generated disassembly sequences are always valid. As shown in Fig. 4(a), suppose that the position of task 5 needs to be changed. Currently, it is located at position 5, and it has tasks 1 and 8 as its immediately preceding task and immediate successor tasks respectively, occupying positions 1 and 7 respectively. Since the neighborhood structure is left insert structure, task 5 can only be inserted at any position between 2 and 4.

Right single-point insert neighborhood structure: Change the position of only one task, but the new position of the task can only be to the right of the original position. First, randomly select a position of the food source, then get the preceding task and the following task of the task at this position according to the P . This task can be inserted into any position between the preceding task and the following task, while the new position should be on the right of the original position. Similarly, if the position of task 5 is to be changed, its viable range of displacement is constrained to be between positions 4 and 6.

Multi-point inserts neighborhood structure: Change the position of multiple tasks, the operation complexity is larger than that of single-point insertion but usually smaller than range transformation. Multi-point insert neighborhood structure adjusts the position of several tasks multiple times, and the steps for each adjustment are the same as single-point insertion.

Scope-reconstruct neighborhood structure: Regenerating the sequence of tasks between any two points in a food source, which is the neighborhood structure with the greatest operational complexity. Two points (p_1 and p_2) on the food source are randomly selected, the P is updated, only the priority relationship between the two points remains, and a new disassembly sequence is generated according to the updated P . Due to the reconstruction based on the precedence matrix, the new task sequence always satisfies the precedence constraints, ensuring its validity.

6.4. Employed bee phase

In the employed bee phase, the employed bee conducts a preliminary exploration on each subpopulation. To speed up the search process, the APNS strategy is used for exploration. The maximum search number for each solution is set as **uplimit**. If the new solution Pareto dominates the old one, replace the old one. Otherwise, the number of unimproved trials increases by one. When the number of unimproved trials exceeds the specified **uplimit**, the solution is deemed to have no further exploration value, and the search for that solution is discontinued. Algorithm 5 describes the search process of the employed bee phase.

Algorithm 5: The Employed Bee Phase

Input: solution set X and corresponding fitness value $f(X)$
Output: a new solution set X_2

```

1  for each solution  $x_i$  of the solution set  $X$  do
2    for i = 0 to uplimit do
3      New solution  $x_j \leftarrow$  APNS Strategy.
4      if  $f(x_j)$  Pareto dominates  $f(x_i)$  do
5         $x_i = x_j$ , trial( $x_i$ ) = trial( $x_i$ ) + 1.
6      end if
7    end for
8  end for
9  Return  $X_2$ 

```

6.5. Onlooker bee phase

In the onlooker bee phase, the onlooker bee selects some food sources through the tournament for further exploration. Since the optimal solutions of the past dynasties are stored in the external

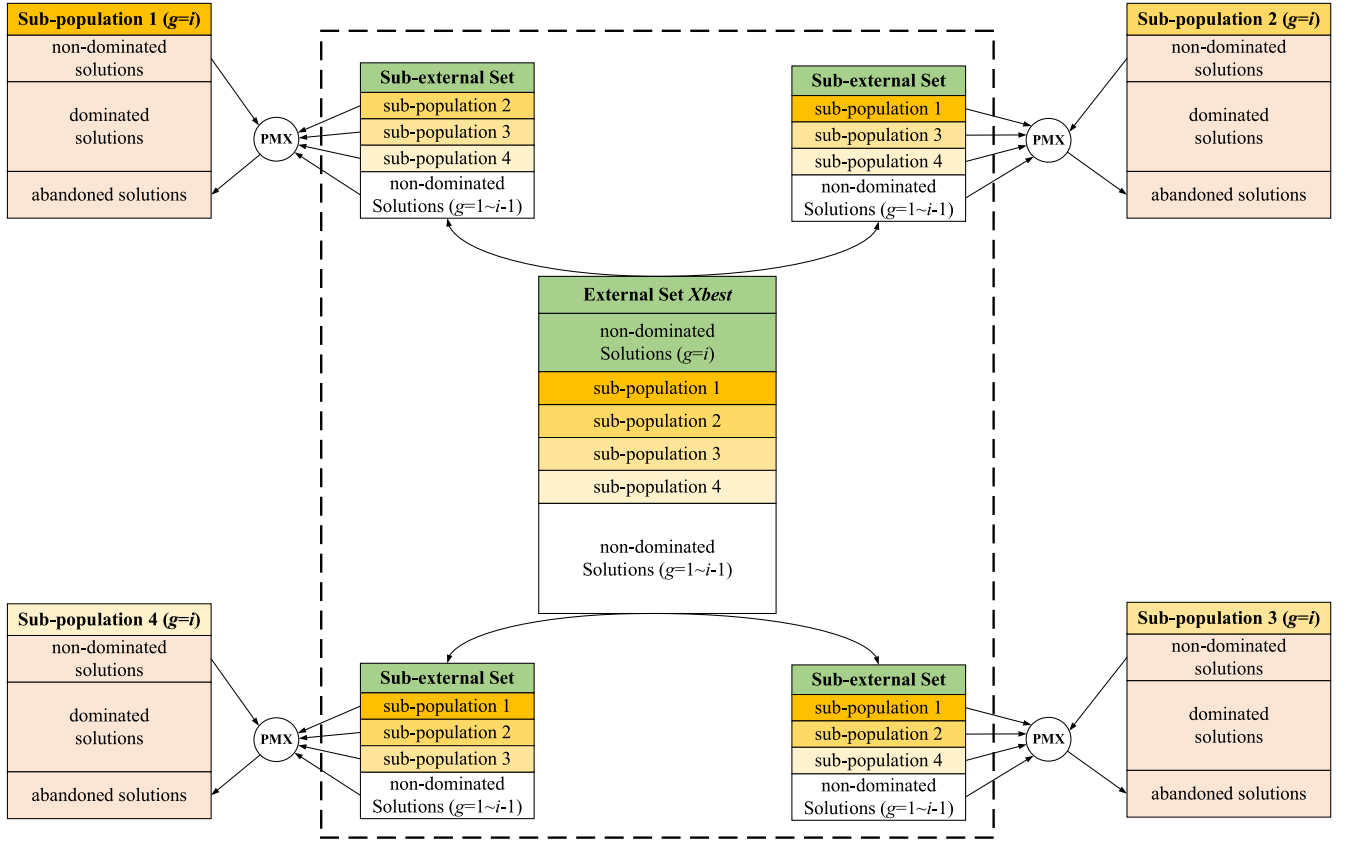


Fig. 5. The flow chart of the CCHI strategy.

archives, selecting the solutions in the external archives for exploration may generate a better scheduling plan, so some observers will select the solutions in the external archives for exploration. The search process in the onlooker bee phase is shown in Algorithm 6.

Algorithm 6: The Onlooker Bee Phase

Input: solution set X , external archives $Xbest$, and corresponding fitness value $f(X)$ and $f(Xbest)$
Output: a new solution set X_2

- 1 Randomly select several solutions from the external archives $Xbest$ and add them to X to get X_2 , then sort and select X_2 in ascending order of Pareto dominance to ensure the number of X_2 is equal to X
- 2 **for** each solution x_i of the solution set X_2 **do**
- 3 **while** $trial(x_i) < uplimit$ **do**
- 4 New solution $x_j \leftarrow APNS$ Strategy.
- 5 **if** $f(x_j)$ Pareto dominates $f(x_i)$ **do**
- 6 $x_i = x_j$, $trial(x_i) = 0$.
- 7 **else**
- 8 $trial(x_i) = trial(x_i) + 1$.
- 9 **end if**
- 10 **end while**
- 11 **end for**
- 12 **Return** X_2

subpopulations to generate new solutions, thus promoting information exchange between populations and increasing population diversity. The pseudo-code of the CCHI strategy is shown in Algorithm 7.

Algorithm 7: The CCHI strategy

Input: subpopulations 1–4
Output: new subpopulations 1–4

- 1 Classify the individuals in each subpopulation into dominated, non-dominated, and abandoned solutions, and then the non-dominated solutions from each subpopulation are stored in the external set $Xbest$.
- 2 **for** $sub = 1$ to 4 **do**
- 3 The non-dominated solutions from the remaining subpopulations, along with the non-dominated solutions from history generations, are merged into a new set, creating a sub-external set.
- 4 **for** each solution x_i of subpopulation sub **do**
- 5 **while** $trial(x_i) > uplimit$ **do**
- 6 Randomly select another individual x_j from the sub-external set.
- 7 The solution x_i and solution x_j generate a new solution x through PMX.
- 8 update solution x_i using solution x .
- 9 **end while**
- 10 **end for**
- 11 **end for**
- 12 **Return** new subpopulations 1–4

6.6. Scout bee phase based on CCHI strategy

Subpopulations coevolve through information interaction. In the process of subpopulation information interaction, the information exchange is determined by scout bees. In the scout bee phase, new food sources will be generated to replace those that have reached the *uplimit*, i.e., abandoned solutions. In contrast to the traditional ABC algorithm where new solutions are randomly generated, this study proposes a CCHI strategy by exchanging information from the best solution within each population and the historical best solution from other

The flow chart of the CCHI strategy is shown in Fig. 5. For the i -th generation population, the details of the CCHI strategy are as follows.

Step 1: Classify the individuals in each subpopulation into dominated, non-dominated, and abandoned solutions, and then the non-dominated solutions from each subpopulation are stored in the external set $Xbest$.

Step 2: Using subpopulation 1 as an example, as depicted in Fig. 5, aggregate the non-dominated solutions from other subpopulations (i.e., subpopulation = 2–4) in the current generation or the historical nondominated solutions (i.e., $g = 1 \sim i$) to form a sub-external set.

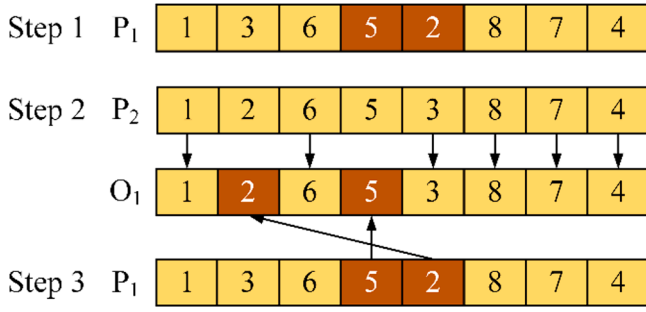


Fig. 6. PMX operation.

Table 1
The time complexity of MPCCABC.

	sub-function	time complexity
Initialization phase	Initialization population	$O(np)$
	Evaluate the fitness value	$O(np)$
Iterative phase	Rank the population	$O((np)^2)$
	Employed Bee Phase (APNS)	$O(np \times uplimit \times n)$
	Onlooker Bee Phase (APNS)	$O(np) + O(np \times uplimit \times n)$
	Scout Bee Phase (CCHI)	$O(np) + O((np)^2) + O(k \times n)$
	Rank the population	$O((np)^2)$
	Total time complexity	$O(np \times (np + maxTime \times max(n, np)))$

Step 3: Randomly select one parent from both subpopulation 1 and the sub-external set for the PMX operation.

Step 4: The two selected individuals generate the new solution by PMX.

The PMX is shown in Fig. 6. The steps of the PMX operation are as follows:

Step 1: Randomly select 2 crossover points on parent 1 (P₁), and the segment between the two crossover points is the crossover task, denoted as {5,2}.

Step 2: Copy all genes of parent 2 (P₂) to offspring 1 (O₁).

Step 3: Copy the genes contained in the crossover task of P₁ to offspring 1, while maintaining their positions in P₁.

Additionally, to ensure both the quality and diversity of new solutions, in addition to the CCHI strategy, random and heuristic rule-based solution generation methods are employed, as elaborated in Section 3.2.

6.7. Computational complexity

Computational complexity directly reflects an algorithm's running time [54]. Therefore, detailing the time complexity of the MPCCABC algorithm is crucial. Table 1 shows the time complexity of sub-functions of the MPCCABC algorithm, assuming k individuals exceed $uplimit$ in each subpopulation, where $k \leq np$.

According to the worst-case analysis and asymptotic analysis methods, the total time complexity of MPCCABC is calculated as $O(np \times (np + maxTime \times max(n, np)))$. Therefore, the time complexity of the MPCCABC algorithm depends on the population size, the maximum number of iterations, and the dimension of the proposed problem.

7. Experiments and result analysis

In order to assess the effectiveness of the MPCCABC, this section employs three different scales of problem instances. The performance

Table 2

The results of the Taguchi experiment.

No.	Levels of parameters				Average HVR		
	np	$maxTime$	$uplimit$	θ	Case 1	Case 2	Case 3
1	25	50	5	0.2	0.6834	0.77550	0.3499
2	25	100	10	0.4	0.7233	0.91620	0.3117
3	25	150	15	0.6	0.7942	1.05066	0.2817
4	25	200	20	0.8	0.7846	1.10308	0.4325
5	50	50	10	0.6	0.7127	0.94840	0.4521
6	50	100	5	0.8	0.7279	1.00454	0.5505
7	50	150	20	0.2	0.8944	0.89741	0.4235
8	50	200	15	0.4	0.7838	0.90367	0.2965
9	75	50	15	0.8	0.8975	1.16310	0.3772
10	75	100	20	0.6	0.9423	1.14414	0.2539
11	75	150	5	0.4	0.7799	0.97682	0.5228
12	75	200	10	0.2	0.7930	0.89864	0.8855
13	100	50	20	0.4	0.9030	1.03601	0.3738
14	100	100	15	0.2	0.8449	0.78952	0.3183
15	100	150	10	0.8	0.8218	1.17516	0.3218
16	100	200	5	0.6	0.8380	1.09524	0.2446

evaluation of the algorithms in this study is based on the hyper-volume ratio (HVR) and inverted generational distance (IGD). All experiments were conducted on a computer system equipped with an Intel (R) Core (TM) i5 CPU operating at 3.90 GHz and 8.00 GB of RAM.

7.1. Evaluation criteria

The behavior of algorithms is evaluated by IGD and HVR. IGD is believed as a reliable performance evaluator to quantify concurrently the convergence and diversity of multi-objective algorithms. A smaller IGD value means better convergence and higher diversity. The calculation formula for IGD is shown in Eq. (17).

$$IGD(S) = \frac{\sqrt{\sum_{k=1}^{|TP|} dt_k^2}}{|TP|} \quad (17)$$

where $|TP|$ means the number of solutions in the true Pareto front (TP) and dt_k is the Euclidean distance between solution k in TP and the nearest point in the obtained Pareto front set (S).

HVR measures the volume of the dominated space. It indicates the distribution of solutions over the Pareto front and hence is capable of serving as comprehensive metrics. A higher HVR value reflects better comprehensive performance. The calculation formula for HVR is shown in Eq. (18).

$$HVR(S) = \frac{\text{volume}(U_{i=1}^n v_i)}{\text{volume}(U_{j=1}^m v_j)} \quad (18)$$

where n and m represent the number of individuals obtained by an algorithm and the true Pareto front. v_i refers to the i -th hypercube, whose diagonal corners are the objective vector of solution i in the obtained Pareto set and that of the reference point.

7.2. Parameter calibration

The parameter combination of each algorithm is determined via the Taguchi experiment, with 10 repetitions for each combination. For the proposed MPCCABC, four parameters need to be calibrated, which are np , $maxTime$, $uplimit$, and θ . HVR is selected as the appropriate scalar indicator due to it combines both the distance of solutions and the spread of solutions [55]. Table 2 shows the levels of these parameters and the average HVR of 10 times experiments. It can be seen from Fig. 6 that $\{np 75; maxTime 150; uplimit 20; \theta 0.6\}$, $\{np 75; maxTime 150; uplimit 20; \theta 0.8\}$, and $\{np 75; maxTime 200; uplimit 10; \theta 0.2\}$ are the optimal parameter combination of MPCCABC in three cases, respectively.

Table 3

The best parameter combination of tested algorithms.

Algorithms	Parameters	Ranges	Value		
			Case 1	Case 2	Case 3
ABC	<i>np</i>	25,50,75,100	50	100	100
	<i>uplimit</i>	5,7,9,11	9	7	11
	<i>maxTime</i>	50,100,150,200	100	100	150
ABC-MPIS	<i>np</i>	25,50,75,100	50	100	75
	<i>uplimit</i>	5,7,9,11	9	7	9
	<i>maxTime</i>	50,100,150,200	100	100	150
ABC-APNS	<i>np</i>	25,50,75,100	55	25	25
	<i>uplimit</i>	5,7,9,11	9	5	11
	<i>maxTime</i>	50,100,150,200	100	100	150
ABC-CCHI	<i>np</i>	25,50,75,100	100	75	100
	<i>uplimit</i>	5,7,9,11	11	7	9
	<i>maxTime</i>	50,100,150,200	200	100	100
	θ	0.2 0.4 0.6 0.8	0.6	0.8	0.8
NSGA-II	<i>np</i>	25,50,75,100	75	75	75
	crossover rate	0.2,0.4,0.6,0.8	0.8	0.8	0.8
	mutation rate	0.2,0.4,0.6,0.8	0.2	0.2	0.4
	<i>maxTime</i>	25,50,75,100	75	75	100
PSO	<i>np</i>	50,100,150,200	75	100	75
	<i>maxTime</i>	25,50,75,100	100	100	150
SA	<i>np</i>	25,50,75,100	75	100	75
	initial temperature	25,50,75,100	100	100	100
	Cooling rate	0.85,0.89,0.93,0.97	0.97	0.97	0.97

The parameters for the other algorithms are adjusted using the consistent calibration approach and their values are reported in Table 3.

7.3. Validation of the mathematical model

In this section, the software CPLEX 12.8.0 is used to verify the correctness of the proposed model. And, the ϵ -constraint method [56] is used due to four objectives that need to be optimized. Since the true Pareto front cannot be obtained within a few hours by CPLEX for solving large-scale scenarios, a 20-part case [57] is used to test the correctness of the proposed PMRDLP-PM mode. In this case, there are two workstations and four robots. Besides, there are two scenarios in this example: one regular working scenario and a PM scenario where robot 1 is under PM. Table 4 shows the optimal solutions obtained by MPCCABC and CPLEX. It can be seen that the MPCCABC algorithm yields optimal solutions that match the exact solutions obtained through the utilization of the CPLEX, which verifies the correctness of the proposed model. Furthermore, the solving time for MPCCABC is less than 20 s, while CPLEX exceeds 40 min. Hence, it is imperative to develop MPCCABC to address the problem effectively.

7.4. Case study

7.4.1. Case study 1

The first case is an iPhone 6 case with 20 parts [57], and the specific data are presented in the Appendix. In this case, there are two

workstations and four robots in the disassembly line. Robot 1 needs to be maintained. Table 5 shows the best values of all algorithms. Fig. 7 represents the four-dimensional spatial distribution of the Pareto front of the four algorithms. In Fig. 7, the fourth dimension is represented by the color bar depicting *R*, while the three-dimensional coordinates represent *CT*, *E*, and *Pf*. The four single-objective nearly optimal values obtained by NSGA-II, PSO, SA, and MPCCABC are {210, 212.62, 426.92, 8}, {210, 213.71, 403.0, 4}, {222, 213.71, 403.73, 5}, and {210, 213.71, 403.0, 4}. Obviously, MPCCABC can achieve the best value compared with other algorithms in terms of target cycle time (*CT*), total energy consumption (*Pf*), profit (*E*), and the cost of task reassignment (*R*).

Table 6 illustrates the average (AVG) and best values of *IGD* and *HVR* obtained by all algorithms. The MPCCABC algorithm attains the highest *HVR* values for both average (0.6189) and best (0.6952) cases, surpassing NSGA-II, PSO, and SA. Additionally, MPCCABC exhibits the lowest *IGD* values in both average (0.0602) and best (0.0577) cases, respectively, outperforming SA, PSO, and NSGA-II. Fig. 8 presents the box plot of *HVR* and *IGD*. These results unequivocally show the superior performance of the MPCCABC in the iPhone 6 case.

7.4.2. Case study 2

The second instance is a printer with 55 parts [46]. In this case, there are three workstations and four robots in the disassembly line. Robot 2 needs to be maintained. Table 7 shows that the proposed MPCCABC algorithm achieves the best value among the four algorithms across all functions *CT*, *Pf*, *E*, and *R*. Fig. 9 represents the four-dimensional spatial distribution of the Pareto front of the four algorithms. The four single-objective nearly optimal values obtained by NSGA-II, PSO, SA, and MPCCABC are {286, 88.79, 849.7, 30.11}, {295.6, 127.63, 562.97, 30.86}, {287.64, 128.63, 588.24, 33.37} and {285.87, 128.73, 586.56, 38.43}. Obviously, MPCCABC outperforms the other three algorithms.

Table 8 presents values of *IGD* and *HVR* obtained by all methods. MPCCABC algorithm attains the highest *HVR* values for both average (0.4337) and best (0.4717) cases, surpassing PSO, SA, and NSGA-II. Additionally, MPCCABC exhibits the lowest *IGD* values in both average (0.0595) and best (0.0496) cases, outperforming PSO, SA, and NSGA-II. Fig. 10 presents the box plot of *IGD* and *HVR*. These results unequivocally show the superior performance of the MPCCABC in the printer case.

7.4.3. Case study 3

As electric vehicles (EVs) become more and more popular, their

Table 5

The best value obtained by all comparison algorithms.

Algorithm	<i>CT</i>	<i>Pf</i>	<i>E</i>	<i>R</i>
MPCCABC	210	213.71	403.0	4
NSGA-II	210	212.62	426.4	8
PSO	222	213.71	403.7	5
SA	217	213.71	413.3	17

Table 4

The solutions in the Pareto front obtained by MPCCABC and CPLEX.

Pareto solution		CPLEX				MPCCABC			
regular working scenario	robot 1 under PM scenario	f_1	f_2	f_3	f_4	f_1	f_2	f_3	f_4
W1 = {R4{1,2}, R1{4,5,6,9}, R3{13,14}} W2 = {R2{7,8,10,11,16}}	W1 = {R3{2,1,9,4,14,3,7}} W2 = {R2{8,16,11}, R4{10,13}}	210	182.7	430.4	9	210	182.7	430.4	9
W1 = {R4{1,2}, R1{4,5,6,9}, R3{13,14}} W2 = {R2{7,8,10,11,16}}	W1 = {R4{1,2}, R3{3,4,9,14}} W2 = {R2{7,8,10,11,13,16}}	212	182.6	431.7	4	212	182.6	431.7	4
W1 = {R4{1,2}, R1{4,5,6,9}, R3{13,14}} W2 = {R2{7,8,10,11,16}}	W1 = {R4{1,3,4,7,9}, R2{2}} W2 = {R3{8,10,11,13,14,16}}	240	205	403	12	240	205	403	12
W1 = {R4{1,2}, R1{4,5,6,9}, R3{13,14}} W2 = {R2{7,8,10,11,16}}	W1 = {R4{1,2,5,6}, R3{12,13,14,15}} W2 = {R2{4,7,8,10,11,16}}	262	213.7	478.0	5	262	213.7	478.0	5

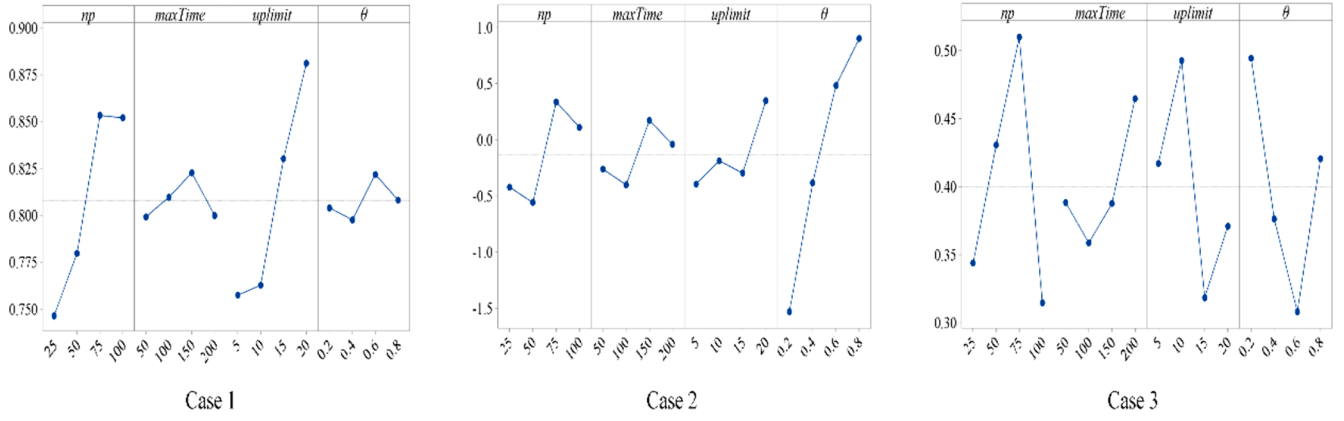


Fig. 7. Signal-to-noise ratio main effect map.

Table 6

The results of *HVR* and *IGD* of all comparison algorithms.

Algorithm	<i>HVR</i>		<i>IGD</i>	
	AVG	BEST	AVG	BEST
NSGA-II	0.3749	0.5632	0.3706	0.3501
PSO	0.5455	0.6395	0.2391	0.2122
SA	0.5472	0.6712	0.1891	0.1796
MPCCABC	0.6189	0.6952	0.0602	0.0577

production is increasing rapidly, which will inevitably bring a large number of the waste battery module disassembly and recycling problems. This section takes the Tesla Model S as an example to verify the effectiveness of the proposed algorithm. The disassembly information of the Tesla Model S can be obtained from [58]. In this case, there are three workstations and four robots in the disassembly line. Robot 4 needs to be maintained.

As shown in Table 9, the MPCCABC is better than the three other algorithms on four objective *CT*, *Pf*, *E*, and *R*. The spatial distribution of

the Pareto front of the four algorithms is shown in Fig. 11. The four single-objective nearly optimal values obtained by NSGA-II, PSO, SA, and MPCCABC are {988, 739.48, 2974.3, 20}, {949, 723.40, 2816.35, 43}, {866, 720.87, 2662.36, 65} and {834, 742.72, 2614.17, 19}. In the Tesla Model S case, MPCCABC clearly performs better than the other three classic algorithms.

Table 10 shows the values of *IGD* and *HVR* obtained by all methods. MPCCABC algorithm attains the highest *HVR* values for both average (0.4149) and best (0.5025) cases, surpassing PSO, SA, and NSGA-II. Additionally, MPCCABC exhibits the lowest average and best *IGD* values for both average (0.3165) and best (0.2563) cases, outperforming

Table 7

The best value obtained by all comparison algorithms.

Algorithm	<i>CT</i>	<i>Pf</i>	<i>E</i>	<i>R</i>
MPCCABC	162	89.12	496.86	12
NSGA-II	273	85.27	831.03	50
PSO	277	85.26	801.69	65
SA	274	84.94	816.95	55

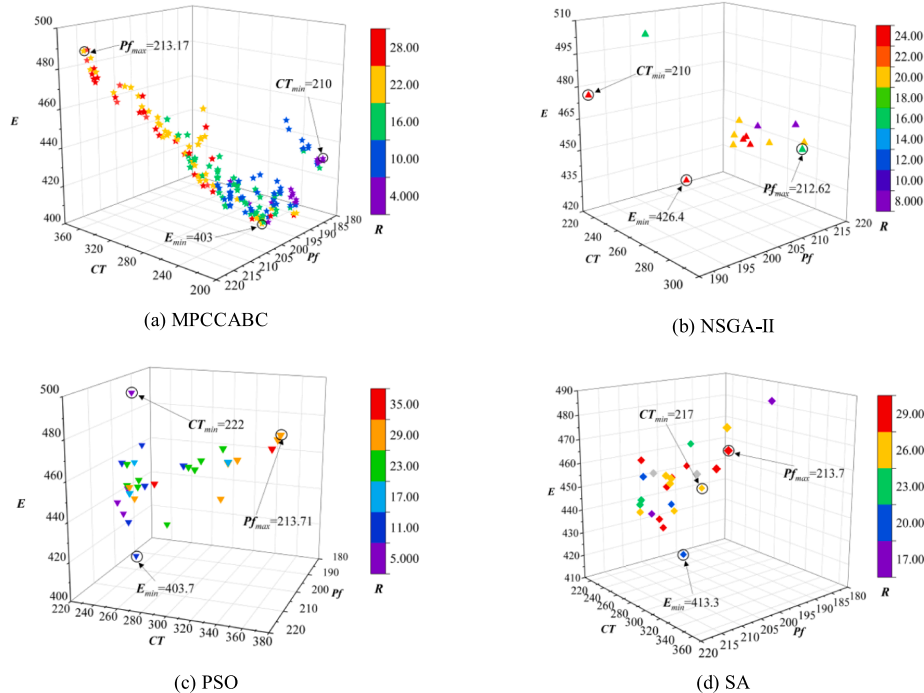


Fig. 8. The Pareto front of four algorithms.

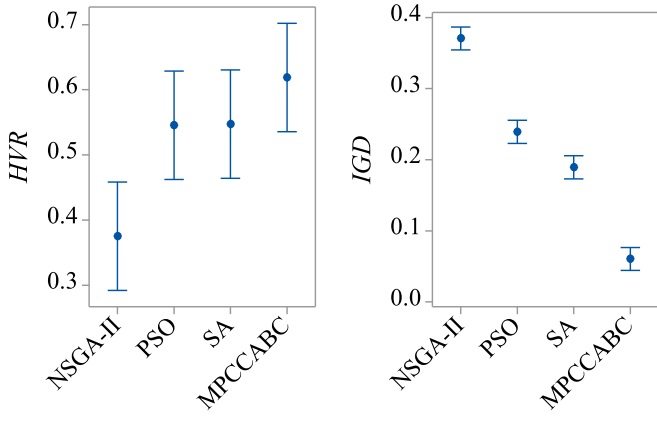


Fig. 9. Boxplot of HVR and IGD on iPhone 6 case.

Table 8

The results of HVR and IGD of all comparison algorithms.

Algorithm	HVR		IGD	
	AVG	BEST	AVG	BEST
NSGA-II	0.1701	0.2832	0.7160	0.7029
PSO	0.2033	0.2642	0.5936	0.5932
SA	0.1674	0.2415	0.6475	0.6130
MPCCABC	0.4337	0.4717	0.0595	0.0496

Table 9

The best value obtained by all comparison algorithms.

Algorithm	f_1	f_2	f_3	f_4
MPCCABC	834	742.72	2614.17	19
NSGA-II	988	739.48	2947.30	20
PSO	949	723.40	2816.35	43
SA	866	720.87	2662.36	65

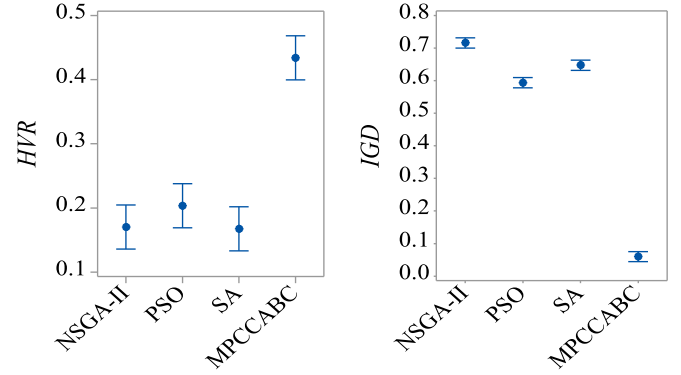
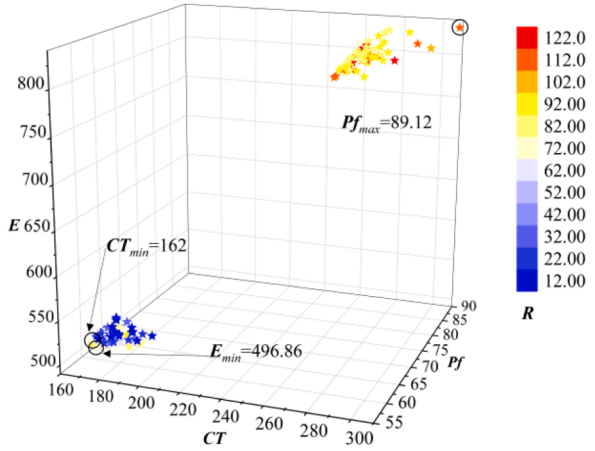
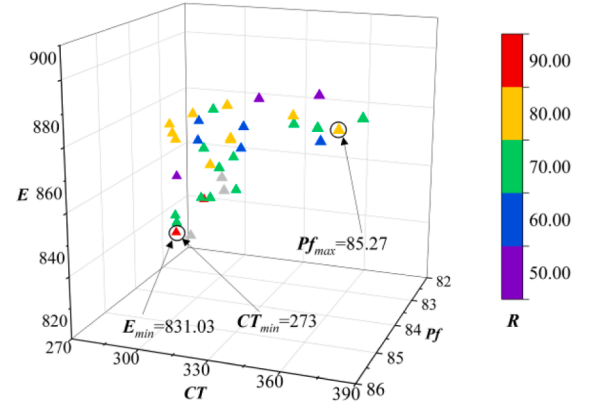


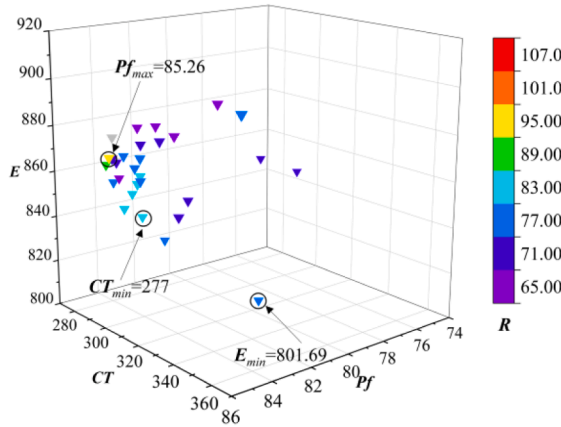
Fig. 11. Boxplot of HVR and IGD on 55-part case.



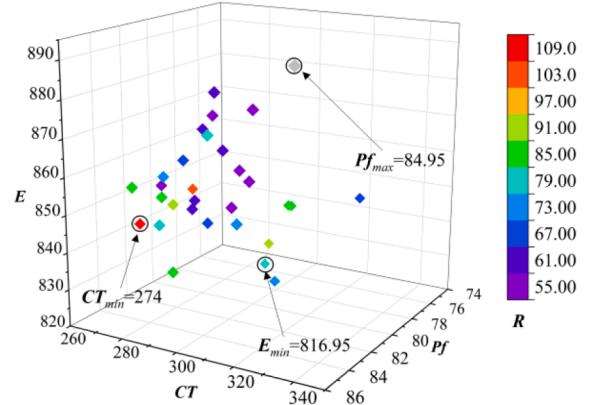
(a) MPCCABC



(b) NSGA-II



(c) PSO



(d) SA

Fig. 10. The Pareto front of four algorithms.

Table 10The results of *HVR* and *IGD* of all comparison algorithms.

Algorithm	<i>HVR</i>		<i>IGD</i>	
	AVG	BEST	AVG	BEST
NSGA-II	0.1158	0.2637	0.6130	0.5763
PSO	0.3419	0.4977	0.5705	0.3790
SA	0.2646	0.4289	0.5923	0.5918
MPCCABC	0.4149	0.5025	0.3165	0.2563

PSO, SA, and NSGA-II. Additionally, Fig. 12 presents the box plot of *IGD* and *HVR*. These results unequivocally show the superior performance of the MPCCABC in the Tesla Model S case.

7.5. Statistical analyses

The Friedman test and the Wilcoxon test with 5 % accuracy are adopted to identify any significant differences between significant differences between all algorithms. Tables 11–13 present the results of the Friedman test and the Wilcoxon test, respectively.

Table 11 presents the statistical results of the Friedman test, indicating that the asymptotic significance values are all below 0.05 across all three cases. This suggests significant differences in the performances of the four algorithms. Furthermore, Friedman rank-based analysis is outlined in this study, transforming the original results into ranks. Ranks are assigned from 1 (best performance) to 4 (worst performance), as explicitly detailed in the paper. This ranking methodology enables comprehensive comparison and analysis of the outcomes.

Table 12 demonstrates that the MPCCABC outperforms all comparison algorithms in terms of ranking the case, which confirms its ability to obtain high-quality solutions.

It can be seen from Table 13 that the results for almost all comparison algorithms and the MPCCABC algorithm yield p-values are less than 0.05, which means that the observed variances are statistically

Table 11

Comparison of Friedman test results.

		DF	Adj. SS	Adj. MS	F-value	P-value
Case1	<i>IGD</i>	3	48.20	16.07	0.57	2.32E–06
	<i>HVR</i>	3	19.4	6.47	0.23	8.70E–03
Case2	<i>IGD</i>	3	40.4	13.47	0.47	2.23E–05
	<i>HVR</i>	3	31.4	10.47	0.37	3.00E–04
Case3	<i>IGD</i>	3	36.2	12.07	0.42	7.45E–05
	<i>HVR</i>	3	37.4	12.47	0.44	5.28E–05

Table 12

The rank of the Friedman test.

		MPCCABC	NSGA-II	PSO	SA
Case 1	<i>HVR</i>	1.7	3.6	2.2	2.5
	<i>IGD</i>	1	3.9	3.1	2
Case 2	<i>HVR</i>	1.3	3.9	2	2.8
	<i>IGD</i>	1	3.6	2.9	2.5
Case 3	<i>HVR</i>	1	3.4	2.5	3.1
	<i>IGD</i>	1	3.8	2	3.2

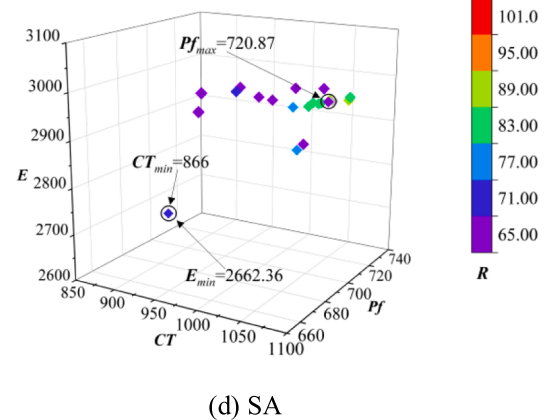
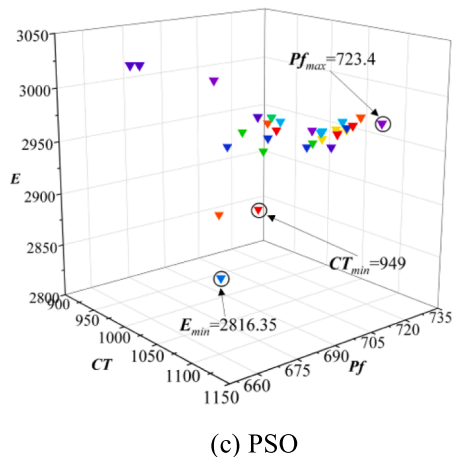
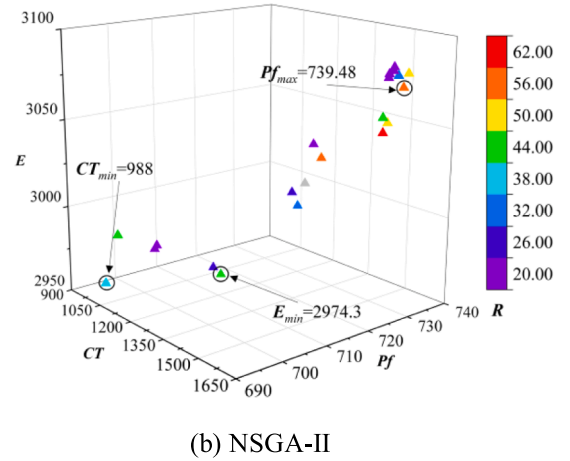
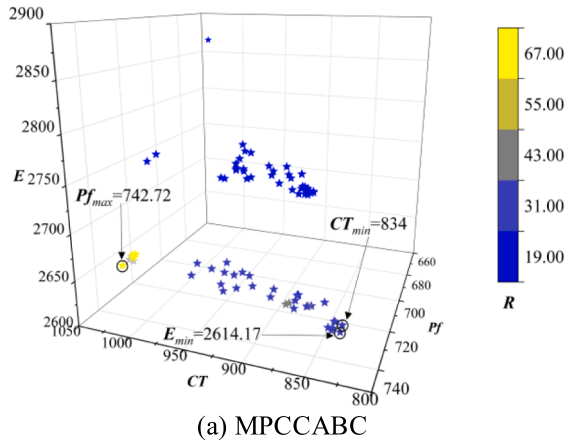
**Fig. 12.** The Pareto front of four algorithms.

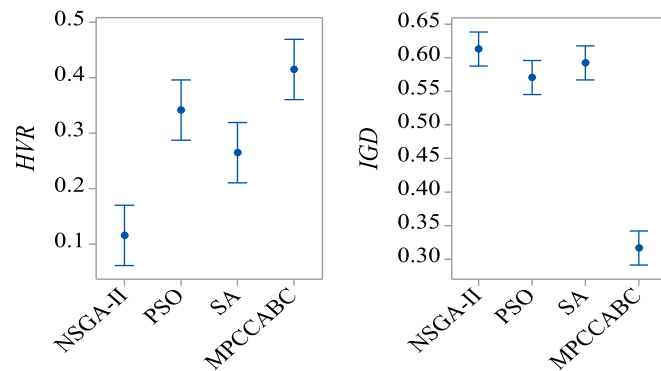
Table 13The results of the Wilcoxon test for MPCCABC versus compared method ($p \geq 0.05$).

Algorithm	Case 1		Case 2		Case 3	
	IGD	HVR	IGD	HVR	IGD	HVR
NSGA-II	1.55E-04	1.31E-03	1.76E-04	1.83E-04	1.65E-04	1.78E-04
PSO	1.56E-04	2.57E-02	1.32E-04	1.83E-04	1.72E-04	7.56E-02
SA	1.73E-04	8.90E-02	1.79E-04	1.83E-04	1.43E-04	1.00E-03

Table 14

The results of IGD and HVR of six algorithms.

			ABC	ABC-MPIS	ABC-APNS	ABC-CCHI	MPCCABC
Case 1	IGD	AVG	0.0723	0.0561	0.0542	0.0516	0.0508
		BEST	0.0606	0.0498	0.0439	0.0448	0.0413
	HVR	AVG	0.4422	0.5521	0.5741	0.7294	0.7773
BEST		0.4553	0.5603	0.6508	0.7523	0.8398	
Case 2	IGD	AVG	0.911	0.8450	0.4887	0.4847	0.2114
		BEST	0.5894	0.6074	0.1128	0.0571	0.1232
	HVR	AVG	0.5766	0.7199	0.8559	0.8683	0.9537
BEST		0.7510	0.7944	0.9901	0.9671	0.9999	
Case 3	IGD	AVG	0.0495	0.0413	0.0416	0.0359	0.0128
		BEST	0.0446	0.0388	0.0340	0.0344	0.0121
	HVR	AVG	0.4888	0.5928	0.5916	0.5812	0.7372
		BEST	0.5160	0.6147	0.6037	0.5843	0.7601

**Fig. 13.** Boxplot of HVR and IGD on Tesla Model S case.

significant. These results indicate that MPCCABC performs better than all other methods evaluated in this study.

7.6. Effectiveness of the proposed strategies in MPCCABC

To test the effectiveness of the three improvements of the proposed algorithm, the multi-population cooperative coevolution artificial bee colony algorithm (MPCCABC) is compared with four algorithms, including the basic ABC without the multi-population initialization strategy (ABC), the basic ABC in which the multi-population initialization strategy (ABC-MPIS), the basic ABC with the adaptive progressive neighborhood search strategy (ABC-APNS), the basic ABC with the cooperative co-evolution strategy with historical information (ABC-CCHI), and the MPCCABC. Each algorithm is run 10 times on three cases. The results are reported as shown in Table 14 and Fig. 13.

It can be observed that when the multi-population initialization strategy is employed, the algorithm performs better in terms of IGD and HVR compared to the original ABC algorithm. This indicates the effectiveness of the heuristic initialization rules designed for each subpopulation. Additionally, it can be observed that when the proposed APNS strategy and CCHI strategy are utilized, the performance of the

algorithm is enhanced. Moreover, with the simultaneous use of all three improvements, MPCCABC achieves the best values for both indicators, further demonstrating the effectiveness of the proposed enhancements (see Fig. 14).

8. Conclusion and future research

In the current robotic disassembly line balancing research, it is assumed that the robot is always available, and the influence of the robot state on the disassembly line is ignored. And the existing scheduling scheme cannot be implemented when the robot fails or needs maintenance, the production line has to shut down, and the production capacity is wasted. Hence, the PMRDLBP-PM is proposed and studied to ensure the stability of the production line. Subsequently, a multi-objective optimization mathematical model is constructed, with the objectives of minimizing production cycle time, disassembly profit, robot energy consumption, and task reassignment. Additionally, a novel MPCCABC with three improvements is proposed to solve the problem. To maintain the diversity of the population, the population is divided into four subpopulations. And, specific designed initialization rules for the optimization objectives of the problem are designed for each subpopulation to enhance the quality of the initial population. To promote search efficiency, an adaptive progressive neighborhood search strategy is proposed, which selects the neighborhood structure with appropriate operational complexity based on the detection results. To enhance the utilization of excellent genes, the cooperative co-evolution strategy with historical information is employed for the elimination of unimproved solutions. Finally, the performance of MPCCABC is evaluated using three different scales of cases and compared with three classical algorithms. Based on the computational results, it can be observed that the three proposed improvement strategies are effective, and the proposed MPCCABC demonstrates superior performance in terms of IGD and HVR.

In this paper, we assume that all robots can independently perform each disassembly task. In the actual disassembly process, however, robots may encounter challenges with complex or damaged components, necessitating human assistance. Therefore, future investigations could explore human-robot collaborative disassembly modes. Additionally, this study focuses on the disassembly line balancing problem for a single product. Given the diversity in disassembly requirements, concurrently disassembling multiple products would likely better align with the needs of enterprises. Hence, a future research direction could involve the mixed-flow disassembly of multiple products.

CRedit authorship contribution statement

Jun Guo: Writing – review & editing, Methodology, Conceptualization. **Yang Li:** Writing – original draft, Software. **Baigang Du:** Supervision, Investigation, Data curation. **Xiang Sun:** Visualization, Validation. **Kaipu Wang:** Resources, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

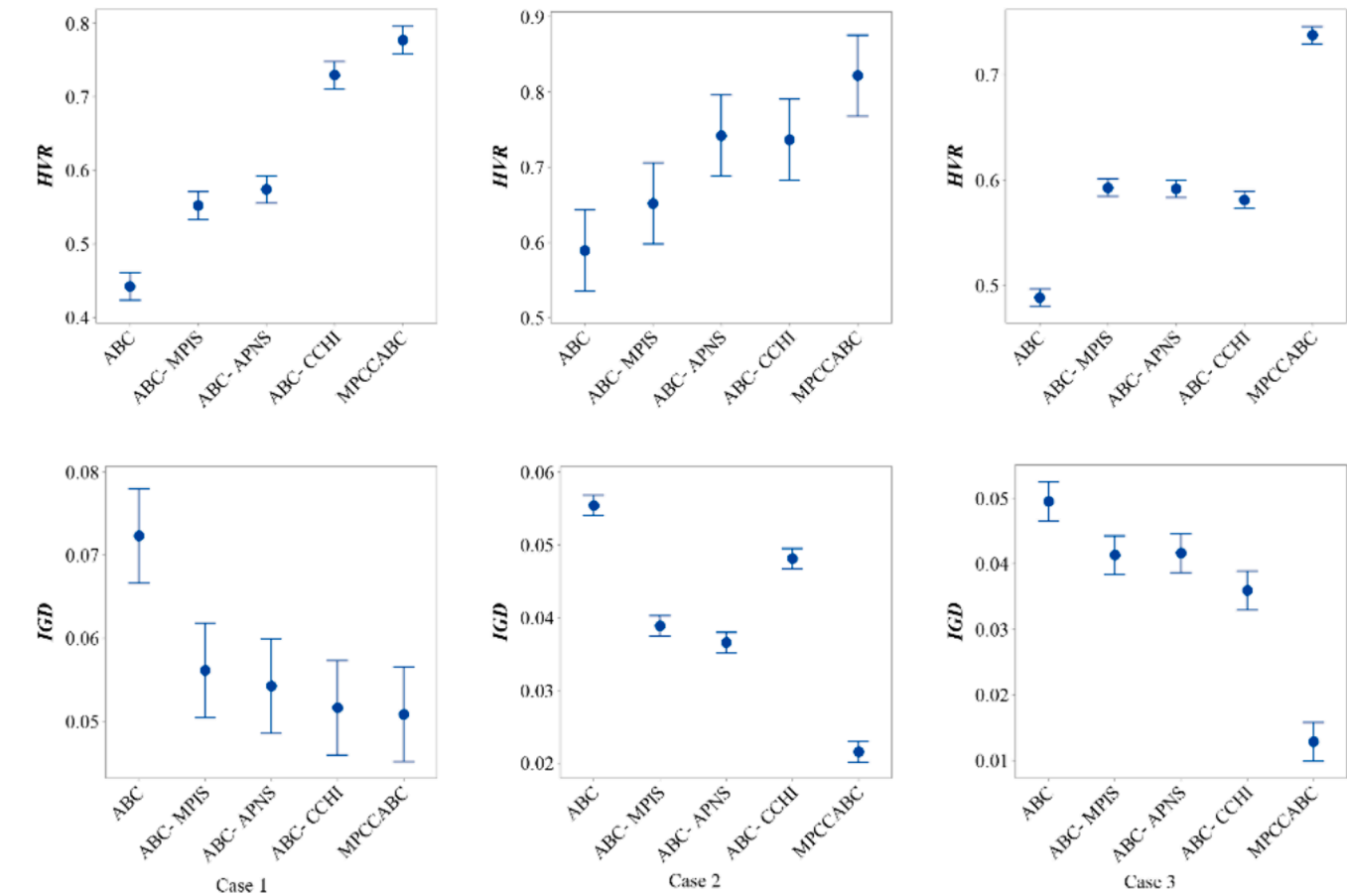


Fig. 14. 95 % confidence interval for the average IGD and HVR of five algorithms in three cases.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Project (52305552, 51705386); China Scholarship Council (201606955091).

Appendix A

Disassembly information of cases iPhone 6, printer, and Tesla Model S.

Table A.1
20 Disassembly tasks of the iPhone 6 instance.

Task No.	Removed parts	Task No.	Removed parts
1	Screen	11	Mainboard
2	Screw 1	12	Dock Connector Cable
3	Earpiece	13	Connector
4	Screw 2	14	SIM Tray
5	Home Button	15	Vibrator
6	Heat Sink	16	Speaker
7	Front Camera	17	Screw 4
8	Screw 3	18	Power Cable
9	Battery	19	Volume Cable
10	Rear Camera	20	Back Cover

Table A.2

Data information of the iPhone 6 instance.

Task No.	re_i	d_i	h_i	P	Task No.	re_i	d_i	h_i	P
1	80	1	0	3,4,5,6,7,8,9,10,11,12, 13,14,15,16,17,18,19,20	11	65	0	0	17,18,19,20
2	0.05	0	0	1	12	6.5	0	0	17,20
3	3	0	1	20	13	0.1	0	0	11,17,20
4	0.3	0	0	3,5,6,7,20	14	5	0	0	11,20
5	1.5	0	0	20	15	5	0	0	20
6	4.5	0	0	20	16	5	0	0	20
7	28	1	0	20	17	0.27	0	0	18,19,20
8	0.75	0	0	10,11,12,13,14,15, 16,17,18,19,20	18	0.6	0	0	20
9	22	0	1	17,20	19	0.4	0	0	20
10	40	1	0	11,17,20	20	0.53	1	0	0

Table A.3

Disassembly time and tools for different robots used in the iPhone 6 instance.

Task No.	r_1	r_2	r_3	r_4	T	Task No.	r_1	r_2	r_3	r_4	T
1	35	38	40	39	3	11	28	30	34	32	3
2	7	8	10	9	1	12	14	16	18	17	3
3	3	4	5	4	2	13	2	3	4	3	2
4	45	48	52	50	1	14	3	4	5	4	4
5	4	5	6	5	2	15	3	4	5	4	4
6	5	6	8	7	2	16	5	6	8	7	3
7	4	5	6	5	2	17	20	22	24	26	1
8	150	158	170	164	1	18	16	18	20	19	2
9	20	24	28	26	3	19	10	12	14	13	2
10	3	4	5	4	2	20	2	3	4	3	2

Note: The disassembly powers of the four robots are 1.5 kW, 1.2 kW, 0.8 kW, and 1.0 kW, and the standby power and tool replacement power are 10 % and 80 % of the disassembly power, respectively. The tool replacement time of the four robots is 3 s, 3 s, 2 s, and 4 s. The unit time cost of the robot of the four robots is 0.05, 0.03, 0.015, and 0.02.

Table A.4

55 Disassembly tasks of the printer disassembly instance.

Task No.	Removed parts	Task No.	Removed parts
1	Power cord	29	Pickup assembly
2	Toner cartridge	30	Take line 2
3	Paper roll	31	Gear dust cover screw
4	Paging device screw	32	Gear dust cover
5	Paging device	33	Relay screw
6	Input tray	34	Relay
7	Front panel	35	Formatted plate screw
8	Left plate screw	36	Format board
9	Left side panel	37	Drive gear set
10	Right side plate screw	38	Spring group
11	Right side plate	39	ECU cover screw
12	Pull rod	40	ECU cover
13	Top cover screw	41	Base screw
14	Cover	42	Base
15	Cable group	43	Engine screw
16	Heating assembly screw	44	Engine
17	Heating component	45	Right medial plate screw
18	Screw back	46	Right medial plate
19	Back	47	Laser circuit board screw
20	Power plate screw 1	48	Laser circuit board cable 1
21	Power plate screw 2	49	Laser circuit board cable 2
22	Power supply board	50	Laser circuit board
23	Plastic block	51	Laser screw
24	Take Line 1	52	Laser with cable
25	Ground	53	Laser
26	Pickup assembly screw 1	54	The left inner plate screw
27	Pickup assembly screw 2	55	The left inner plate
28	Pickup assembly line		

Table A.5

Data information of the printer disassembly instance.

Task No.	re_i	d_i	h_i	P	Task No.	re_i	d_i	h_i	P
1	0.0029	0	0	0	29	0.0053	0	0	25,26,27,28
2	0.0029	0	0	0	30	0.0027	0	0	11
3	0.0027	1	0	0	31	0.0103	0	0	11
4	0.0055	0	0	0	32	0.0029	0	0	30,31
5	0.0033	0	0	4	33	0.0039	0	0	11
6	0.0103	0	0	0	34	0.0027	1	0	33
7	0.0059	0	0	9,11	35	0.0071	0	0	11
8	0.0039	0	0	0	36	0.0027	1	0	35
9	0.0089	0	0	8	37	0.0053	0	0	32
10	0.0039	0	0	0	38	0.0027	0	0	32
11	0.0049	0	0	10	39	0.0087	0	0	7
12	0.0033	0	0	0	40	0.0095	0	0	39
13	0.0055	0	0	0	41	0.0087	0	0	0
14	0.0043	0	0	9,11,12	42	0.0031	0	0	27,41
15	0.0059	0	0	9	43	0.0055	0	0	40
16	0.0071	0	0	14	44	0.0029	1	0	43,48
17	0.0031	1	0	16	45	0.0039	0	0	11
18	0.0039	0	0	0	46	0.0027	0	0	43,45
19	0.0027	0	0	18	47	0.0039	0	0	40
20	0.0055	0	0	9	48	0.0027	0	0	40
21	0.0087	0	0	9	49	0.0027	0	0	40
22	0.0053	0	1	15,20,21, 24,25,28	50	0.0043	0	1	24,30,47,48,49
23	0.0035	0	1	15	51	0.0087	0	0	50
24	0.0027	0	0	9	52	0.0027	0	0	40
25	0.0027	0	0	9	53	0.0027	0	0	49,51,52
26	0.0055	0	0	19	54	0.0039	0	0	22
27	0.0087	0	0	0	55	0.0027	0	0	54
28	0.0031	0	0	9					

Table A.6

Disassembly time and tools for different robots used in the printer disassembly instance.

Task No.	r_1	r_2	r_3	r_4	T	Task No.	r_1	r_2	r_3	r_4	T
1	2	3	4	3	3	29	13	15	17	16	4
2	2	3	4	3	4	30	1	2	3	2	3
3	1	2	3	2	4	31	37	40	43	41	1
4	13	16	19	17	1	32	1	3	5	4	2
5	3	5	8	6	4	33	6	8	10	9	1
6	37	40	43	41	2	34	1	2	3	2	4
7	15	18	21	20	2	35	22	24	26	45	1
8	5	8	11	7	1	36	1	2	3	2	2
9	30	33	36	35	2	37	13	15	17	16	4
10	5	8	11	7	1	38	1	2	3	2	4
11	10	13	16	14	2	39	30	32	34	33	1
12	2	5	8	6	4	40	33	36	39	37	2
13	13	16	19	17	1	41	30	32	34	33	1
14	8	10	12	11	2	42	2	4	6	5	2
15	16	18	20	19	3	43	14	16	18	17	1
16	21	24	26	25	1	44	1	3	5	4	4
17	1	4	7	5	4	45	6	8	10	9	1
18	6	8	10	9	1	46	1	2	3	2	2
19	1	2	3	2	2	47	6	8	10	9	1
20	13	16	19	17	1	48	1	2	3	2	3
21	30	32	34	33	1	49	1	2	3	2	3
22	12	15	18	16	2	50	8	10	12	11	2
23	4	6	8	7	2	51	29	32	35	33	1
24	1	2	3	2	3	52	1	2	3	2	3
25	1	2	3	2	3	53	1	2	3	2	2
26	14	16	18	15	1	54	6	8	10	9	1
27	30	32	34	33	1	55	1	2	3	2	2
28	2	4	6	4	3						4

Note: The disassembly powers of the four robots are 1.5 kW, 1.2 kW, 0.8 kW, and 1.0 kW, and the standby power and tool replacement power are 10 % and 80 % of the disassembly power, respectively. The tool replacement time of the four robots is 3 s, 3 s, 2 s, and 4 s. The unit time cost of the robot of the four robots is 0.05, 0.03, 0.015, and 0.02.

Table A.7

44 Disassembly tasks of the Tesla Model S battery module instance.

Task No.	Removed parts	Task No.	Removed parts
1	Top Cover 1	23	Thermal pad 1
2	Top Cover 2	24	Thermal pad 2
3	Plastic bolt 1	25	Thermal pad 3
4	Plastic bolt 2	26	Thermal pad 4
5	Plastic bolt 3	27	Thermal pad 5
6	Plastic bolt 4	28	Thermal pad 6
7	PCB	29	Thermal pad 7
8	Temperature Sensor	30	Thermal pad 8
9	Routing 1	31	Insulating cushion 1
10	Routing 2	32	Insulating cushion 2
11	Confluence plate 1	33	Insulating cushion 3
12	Confluence plate 2	34	Insulating cushion 4
13	Confluence plate 3	35	Insulating cushion 5
14	Confluence plate 4	36	Insulating cushion 6
15	Confluence plate 5	37	Insulating cushion 7
16	Confluence plate 6	38	Cell group (1)
17	Confluence plate 7	39	Cell group (2)
18	Support frame 1	40	Cell group (3)
19	Support frame 2	41	Cell group (4)
20	Side panel 1	42	Cell group (5)
21	Side panel 2	43	Cell group (6)
22	Cooling pipe	44	Cell group (7)

Table A.8

Data information of the Tesla Model S battery module instance.

Task No.	re_i	d_i	h_i	P	Task No.	re_i	d_i	h_i	P
1	10	0	0	0	23	0.5	0	0	22
2	10	0	0	0	24	0.5	0	0	22
3	0	0	0	1,2	25	0.5	0	0	22
4	0	0	0	1,2	26	0.5	0	0	22
5	0	0	0	1,2	27	0.5	0	0	22
6	0	0	0	1,2	28	0.5	0	0	22
7	150	0	1	3,4,5,6	29	0.5	0	0	22
8	55	0	1	7	30	0.5	0	0	22
9	20	0	1	7	31	0.2	0	0	23,24,25,26, 27,28,29,30
10	20	0	1	7	32	0.2	0	0	23,24,25,26, 27,28,29,30
11	4	0	0	9	33	0.2	0	0	23,24,25,26, 27,28,29,30
12	4	0	0	9	34	0.2	0	0	23,24,25,26, 27,28,29,30
13	4	0	0	9	35	0.2	0	0	23,24,25,26, 27,28,29,30
14	4	0	0	9	36	0.2	0	0	23,24,25,26, 27,28,29,30
15	4	0	0	10	37	0.2	0	0	23,24,25,26, 27,28,29,30
16	4	0	0	10	38	124	1	0	31
17	4	0	0	10	39	128	1	0	32
18	2	0	0	11,15,13,14	40	128	1	0	33
19	2	0	0	15,16,17	41	130	1	0	34
20	1	0	0	18,19	42	130	1	0	35
21	1	0	0	18,19	43	126	1	0	36
22	40	0	0	20,21	44	122	1	0	37

Table A.9

Disassembly time and tools for different robots used in the Tesla Model S battery module instance.

Task No.	r_1	r_2	r_3	r_4	T	Task No.	r_1	r_2	r_3	r_4	T
1	12	15	18	16	1	23	3	5	8	6	3
2	12	15	18	16	1	24	3	5	8	6	3
3	8	10	12	11	2	25	3	5	8	6	3
4	8	10	12	11	2	26	3	5	8	6	3
5	8	10	12	11	2	27	3	5	8	6	3
6	8	10	12	11	2	28	3	5	8	6	3
7	42	45	50	47	3	29	3	5	8	6	3
8	26	30	38	34	3	30	3	5	8	6	3

(continued on next page)

Table A.9 (continued)

Task No.	r_1	r_2	r_3	r_4	T	Task No.	r_1	r_2	r_3	r_4	T
9	55	60	70	65	3	31	12	15	18	16	3
10	55	60	70	65	3	32	12	15	18	16	3
11	90	100	120	110	3	33	12	15	18	16	3
12	90	100	120	110	3	34	12	15	18	16	3
13	90	100	120	110	3	35	12	15	18	16	3
14	90	100	120	110	3	36	12	15	18	16	3
15	90	100	120	110	3	37	12	15	18	16	3
16	90	100	120	110	3	38	180	186	198	192	4
17	90	100	120	110	3	39	186	192	204	198	4
18	28	30	34	32	3	40	186	192	204	198	4
19	28	30	34	32	3	41	189	195	207	201	4
20	18	20	24	22	3	42	189	195	207	201	4
21	18	20	24	22	3	43	183	189	201	195	4
22	102	110	125	118	3	44	177	183	195	189	4

Note: The disassembly powers of the four robots are 1.5 kW, 1.2 kW, 0.8 kW, and 1.0 kW, and the standby power and tool replacement power are 10 % and 80 % of the disassembly power, respectively. The tool replacement time of the four robots is 3 s, 3 s, 2 s, and 4 s. The unit time cost of the robot of the four robots is 0.05, 0.03, 0.015, and 0.02.

References

- [1] Y. Gao, S. Lou, H. Zheng, J. Tan, A data-driven method of selective disassembly planning at end-of-life under uncertainty, *J. Intell. Manuf.* 34 (2021) 565–585, <https://doi.org/10.1007/s10845-021-01812-0>.
- [2] G. Tian, X. Zhang, A.M. Fathollahi-Fard, Z. Jiang, C. Zhang, G. Yuan, D.T. Pham, Hybrid evolutionary algorithm for stochastic multiobjective disassembly line balancing problem in remanufacturing, *Environ. Sci. Pollut. Res.* (2023), <https://doi.org/10.1007/s11356-023-27081-3>.
- [3] J. Guo, Z. Pu, B. Du, Y. Li, Multi-objective optimisation of stochastic hybrid production line balancing including assembly and disassembly tasks, *Int. J. Prod. Res.* 60 (2021) 2884–2900, <https://doi.org/10.1080/00207543.2021.1905902>.
- [4] W. Liang, Z. Zhang, Y. Zhang, Y. Zeng, T. Yin, S. Liu, D. Ji, Improved optimisation method considering full solution space for disassembly line balancing problem in remanufacturing system, *Adv. Eng. Inform.* 58 (2023) 102237, <https://doi.org/10.1016/j.aei.2023.102237>.
- [5] F.F. Zheng, J.K. He, F. Chu, M. Liu, A new distribution-free model for disassembly line balancing problem with stochastic task processing times, *Int. J. Prod. Res.* 56 (2018) 7341–7353, <https://doi.org/10.1080/00207543.2018.1430909>.
- [6] G. Tian, J. Liu, X. Zhang, D.T. Pham, X. Guo, Y. Du, C. Zhao, H. Li, Multi-objective disassembly line design and optimisation considering energy efficiency and human factors, *J. Eng. Des.* (2024), <https://doi.org/10.1080/09544828.2024.2303281>.
- [7] K.L. Keung, C.K.M. Lee, P. Ji, Industrial internet of things-driven storage location assignment and order picking in a resource synchronization and sharing-based robotic mobile fulfillment system, *Adv. Eng. Inform.* 52 (2022) 101540, <https://doi.org/10.1016/j.aei.2022.101540>.
- [8] L. Guo, Z. Zhang, X. Zhang, Human-robot collaborative partial destruction disassembly sequence planning method for end-of-life product driven by multi-failures, *Adv. Eng. Inform.* 55 (2023) 101821, <https://doi.org/10.1016/j.aei.2022.101821>.
- [9] K. Wang, L. Gao, X. Li, P. Li, Energy-Efficient Robotic Parallel Disassembly Sequence Planning for End-of-Life Products, *IEEE Trans. Autom. Sci. Eng.* 19 (2022) 1277–1285, <https://doi.org/10.1109/tase.2021.3061506>.
- [10] Z.K. Zhang, Q.H. Tang, M. Chica, Maintenance costs and makespan minimization for assembly permutation flow shop scheduling by considering preventive and corrective maintenance, *J. Manuf. Syst.* 59 (2021) 549–564, <https://doi.org/10.1016/j.jmsy.2021.03.020>.
- [11] L. Zhao, Q. Tang, Z. Zhang, An improved preference-based variable neighborhood search algorithm with ar-dominance for assembly line balancing considering preventive maintenance scenarios, *Eng. Appl. Artif. Intell.* 109 (2022) 104593, <https://doi.org/10.1016/j.engappai.2021.104593>.
- [12] N. Deniz, F. Ozcelik, Bi-objective optimization-based multi-criteria decision-making framework for disassembly line balancing and employee assignment problem, *Kybernetes* (2023), <https://doi.org/10.1108/k-06-2022-0857>.
- [13] Z.A. Cil, S. Mete, F. Serin, Robotic disassembly line balancing problem: A mathematical model and ant colony optimization approach, *Appl. Math. Model.* 86 (2020) 335–348, <https://doi.org/10.1016/j.apm.2020.05.006>.
- [14] J. Liu, Z. Zhou, P. Duc Truong, W. Xu, J. Yan, A. Liu, C. Ji, Q. Liu, An improved multi-objective discrete bees algorithm for robotic disassembly line balancing problem in remanufacturing, *Int. J. Adv. Manuf. Technol.* 97 (2018) 3937–3962, <https://doi.org/10.1007/s00170-018-2183-7>.
- [15] J. Liu, Z. Zhou, P. Duc Truong, W. Xu, C. Ji, Q. Liu, Collaborative optimization of robotic disassembly sequence planning and robotic disassembly line balancing problem using improved discrete Bees algorithm in remanufacturing, *Robot. Comput.-Integr. Manuf.* 61 (2020) 101829, DOI: 10.1016/j.rcim.2019.101829.
- [16] B. Zhou, J. Bian, Multi-mechanism-based modified bi-objective Harris Hawks optimization for sustainable robotic disassembly line balancing problems, *Eng. Appl. Artif. Intell.* 116 (2022) 105479, <https://doi.org/10.1016/j.engappai.2022.105479>.
- [17] T. Wu, Z. Zhang, S. Ren, L. Guo, Y. Zhang, Y. Zeng, A Hybrid Evolutionary Algorithm for Stochastic Robot Disassembly Line Balancing Problem, in: 2023 29th International Conference on Mechatronics and Machine Vision in Practice (M2VIP), 2023, pp. 1–6, <https://doi.org/10.1109/M2VIP58386.2023.10413367>.
- [18] G. Xu, Z. Zhang, Z. Li, X. Guo, L. Qi, X. Liu, Multi-objective discrete brainstorming optimizer to solve the stochastic multiple-product robotic disassembly line balancing problem subject to disassembly failures, *Mathematics* 11 (2023), <https://doi.org/10.3390/math11061557>.
- [19] K. Wang, Y. Li, J. Guo, L. Gao, X. Li, Dynamic balancing of U-shaped robotic disassembly lines using an effective deep reinforcement learning approach, *IEEE Trans. Ind. Inf.* (2024), <https://doi.org/10.1109/tii.2023.3348811>.
- [20] Y. Fang, Q. Liu, M. Li, Y. Laili, P. Duc Truong, Evolutionary many-objective optimization for mixed-model disassembly line balancing with multi-robotic workstations, *Eur. J. Oper. Res.* 276 (2019) 160–174, <https://doi.org/10.1016/j.ejor.2018.12.035>.
- [21] H. Ming, Q. Liu, P. Duc Truong, Multi-robotic disassembly line balancing with uncertain processing time, in: 11th CIRP Conference on Industrial Product-Service Systems, 2019, pp. 71–76, <https://doi.org/10.1016/j.procir.2019.02.140>.
- [22] Q. Liu, Y. Li, Y. Fang, Y. Laili, P. Lou, P. Duc Truong, Many-objective best-order-sort genetic algorithm for mixed-model multi-robotic disassembly line balancing, in: 11th CIRP Conference on Industrial Product-Service Systems, 2019, pp. 14–21, <https://doi.org/10.1016/j.procir.2019.04.076>.
- [23] Y. Fang, H. Xu, Q. Liu, P. Duc Truong, Evolutionary optimization using epsilon method for resource-constrained multi-robotic disassembly line balancing, *J. Manuf. Syst.* 56 (2020) 392–413, <https://doi.org/10.1016/j.jmsy.2020.06.006>.
- [24] Y. Zhang, Z. Zhang, Y. Zeng, T. Wu, Constraint programming for multi-line parallel partial disassembly line balancing problem with optional common stations, *Appl. Math. Model.* 122 (2023) 435–455, <https://doi.org/10.1016/j.apm.2023.06.009>.
- [25] E.B. Edis, R.S. Edis, M.A. Ilgin, Mixed integer programming approaches to partial disassembly line balancing and sequencing problem, *Comput. Oper. Res.* 138 (2022) 105559, <https://doi.org/10.1016/j.cor.2021.105559>.
- [26] E. Gueler, C.B. Kalayci, M.A. Ilgin, E. Ozceylan, A. Gungor, Advances in partial disassembly line balancing: A state-of-the-art review, *Comput. Ind. Eng.* 188 (2024), <https://doi.org/10.1016/j.cie.2024.109898>.
- [27] B. Chao, P. Liang, C. Zhang, H. Guo, Multi-objective optimization for mixed-model two-sided disassembly line balancing problem considering partial destructive mode, *Mathematics* 11 (2023), <https://doi.org/10.3390/math11061299>.
- [28] L. Guo, Z. Zhang, T. Wu, Y. Zeng, Y. Zhang, X. Xie, A grey wolf optimization algorithm for solving partial destructive disassembly line balancing problem consider feasibility evaluation and noise pollution, *Adv. Eng. Inform.* 60 (2024), <https://doi.org/10.1016/j.aei.2024.102418>.
- [29] N. Durmaz, A. Budak, An integrated Bi-objective green vehicle routing and partial disassembly line problem for electronic waste: an industrial case study, *Int. J. Comput. Integr. Manuf.* (2024), <https://doi.org/10.1080/0951192x.2024.2335984>.
- [30] P. Liang, Y. Fu, K. Gao, Multi-product disassembly line balancing optimization method for high disassembly profit and low energy consumption with noise pollution constraints, *Eng. Appl. Artif. Intell.* 130 (2024), <https://doi.org/10.1016/j.engappai.2023.107721>.
- [31] Z.X. Li, M.N. Janardhanan, Modelling and solving profit-oriented U-shaped partial disassembly line balancing problem, *Expert Syst. Appl.* 183 (2021) 115431, <https://doi.org/10.1016/j.eswa.2021.115431>.
- [32] Y. Zeng, Z. Zhang, T. Yin, H. Zheng, Robotic disassembly line balancing and sequencing problem considering energy-saving and high-profit for waste household appliances, *J. Clean Prod.* 381 (2022) 135209, <https://doi.org/10.1016/j.jclepro.2022.135209>.
- [33] T. Yin, Z. Zhang, Y. Zhang, T. Wu, W. Liang, Mixed-integer programming model and hybrid driving algorithm for multi-product partial disassembly line balancing problem with multi-robot workstations, *Robot. Comput.-Integr. Manuf.* 73 (2022) 102251, <https://doi.org/10.1016/j.rcim.2021.102251>.
- [34] H. Wang, Q. Yan, S. Zhang, Integrated scheduling and flexible maintenance in deteriorating multi-state single machine system using a reinforcement learning

- approach, *Adv. Eng. Inform.* 49 (2021) 101339, <https://doi.org/10.1016/j.aei.2021.101339>.
- [35] Z. Zhang, Q. Tang, X. Qian, Integration of balancing and preventive maintenance in straight and U-shaped resource-dependent assembly lines: MILP model and memetic algorithm, *Appl. Soft. Comput.* 113 (2021) 107773, <https://doi.org/10.1016/j.asoc.2021.107773>.
- [36] Z. Zhang, Q. Tang, D. Han, X. Qian, An enhanced multi-objective JAYA algorithm for U-shaped assembly line balancing considering preventive maintenance scenarios, *Int. J. Prod. Res.* 59 (2021) 6146–6165, <https://doi.org/10.1080/00207543.2020.1804639>.
- [37] S. Mete, Z.A. Cil, E. Celik, E. Ozceylan, Supply-driven rebalancing of disassembly lines: A novel mathematical model approach, *J. Clean Prod.* 213 (2019) 1157–1164, <https://doi.org/10.1016/j.jclepro.2018.12.265>.
- [38] J. Guo, Y. Li, B. Du, K. Wang, Multi-objective fuzzy partial disassembly line balancing considering preventive maintenance scenarios using enhanced hybrid artificial bee colony algorithm, *Expert Syst. Appl.* 237 (2024) 121681, <https://doi.org/10.1016/j.eswa.2023.121681>.
- [39] Y. Zeng, Z. Zhang, T. Wu, W. Liang, Integrated optimization and engineering application for disassembly line balancing problem with preventive maintenance, *Eng. Appl. Artif. Intell.* 127 (2024) 107416, <https://doi.org/10.1016/j.engappai.2023.107416>.
- [40] Y.L. Fang, H. Ming, M.Q. Li, Q. Liu, D.T. Pham, Multi-objective evolutionary simulated annealing optimisation for mixed-model multi-robotic disassembly line balancing with interval processing time, *Int. J. Prod. Res.* 58 (2020) 846–862, <https://doi.org/10.1080/00207543.2019.1602290>.
- [41] S. Mete, Z.A. Cil, E. Ozceylan, K. Agpak, O. Battaia, An optimisation support for the design of hybrid production lines including assembly and disassembly tasks, *Int. J. Prod. Res.* 56 (2018) 7375–7389, <https://doi.org/10.1080/00207543.2018.1428774>.
- [42] Y. Yang, G. Yuan, Q. Zhuang, G. Tian, Multi-objective low-carbon disassembly line balancing for agricultural machinery using MDFOA and fuzzy AHP, *J. Clean Prod.* 233 (2019) 1465–1474, <https://doi.org/10.1016/j.jclepro.2019.06.035>.
- [43] K. Wang, L. Gao, X. Li, A multi-objective algorithm for U-shaped disassembly line balancing with partial destructive mode, *Neural Comput. & Applic.* 32 (2020) 12715–12736, <https://doi.org/10.1007/s00521-020-04721-0>.
- [44] G. Tian, C. Zhang, A.M.M. Fathollahi-Fard, Z. Li, C. Zhang, Z. Jiang, An enhanced social engineering optimizer for solving an energy-efficient disassembly line balancing problem based on bucket brigades and cloud theory, *IEEE Trans. Ind. Inf.* 19 (2023) 7148–7159, <https://doi.org/10.1109/tii.2022.3193866>.
- [45] G. Tian, C. Zhang, X. Zhang, Y. Feng, G. Yuan, T. Peng, D.T. Pham, Multi-objective evolutionary algorithm with machine learning and local search for an energy-efficient disassembly line balancing problem in remanufacturing, *J. Manuf. Sci. Eng.-Trans. ASME* 145 (2023), <https://doi.org/10.1115/1.4056573>.
- [46] H. Guo, L. Zhang, Y. Ren, Y. Li, Z. Zhou, J. Wu, Optimizing a stochastic disassembly line balancing problem with task failure via a hybrid variable neighborhood descent-artificial bee colony algorithm, *Int. J. Prod. Res.* 61 (2022) 2307–2321, <https://doi.org/10.1080/00207543.2022.2069524>.
- [47] K. Wang, X. Li, L. Gao, P. Li, J.W. Sutherland, A discrete artificial bee colony algorithm for multiobjective disassembly line balancing of end-of-life products, *IEEE T. Cybern.* (2021), <https://doi.org/10.1109/tycyb.2020.3042896>.
- [48] Y. Zuo, Z. Fan, T. Zou, P. Wang, A novel multi-population artificial bee colony algorithm for energy-efficient hybrid flow shop scheduling problem, *Symmetry-Basel* 13 (2021) 2421, <https://doi.org/10.3390/sym13122421>.
- [49] G. Tian, M. Zhou, P. Li, Disassembly sequence planning considering fuzzy component quality and varying operational cost, *IEEE Trans. Autom. Sci. Eng.* 15 (2018) 748–760, <https://doi.org/10.1109/tase.2017.2690802>.
- [50] S. Wang, X. Guo, J. Liu, An efficient hybrid artificial bee colony algorithm for disassembly line balancing problem with sequence-dependent part removal times, *Eng. Optimiz.* 51 (2019) 1920–1937, <https://doi.org/10.1080/0305215x.2018.1564918>.
- [51] H. Ma, S. Shen, M. Yu, Z. Yang, M. Fei, H. Zhou, Multi-population techniques in nature inspired optimization algorithms: A comprehensive survey, *Swarm. Evol. Comput.* 44 (2019) 365–387, <https://doi.org/10.1016/j.swevo.2018.04.011>.
- [52] Y. Fu, M. Zhou, X. Guo, L. Qi, K. Sedraoui, Multiverse optimization algorithm for stochastic biobjective disassembly sequence planning subject to operation failures, *IEEE Trans. Syst. Man Cybern.-Syst.* 52 (2022) 1041–1051, <https://doi.org/10.1109/tsmc.2021.3049323>.
- [53] Y. Li, W. Huang, R. Wu, K. Guo, An improved artificial bee colony algorithm for solving multi-objective low-carbon flexible job shop scheduling problem, *Appl. Soft. Comput.* 95 (2020) 106544, <https://doi.org/10.1016/j.asoc.2020.106544>.
- [54] P. Liang, Y. Fu, K. Gao, H. Sun, An enhanced group teaching optimization algorithm for multi-product disassembly line balancing problems, *Complex Intell. Syst.* 8 (2022) 4497–4512, <https://doi.org/10.1007/s40747-021-00478-8>.
- [55] J.M. Nilakantan, Z.X. Li, Q.H. Tang, P. Nielsen, Multi-objective co-operative co-evolutionary algorithm for minimizing carbon footprint and maximizing line efficiency in robotic assembly line systems, *J. Clean Prod.* 156 (2017) 124–136, <https://doi.org/10.1016/j.jclepro.2017.04.032>.
- [56] Q. Tang, K. Meng, L. Cheng, Z. Zhang, An improved multi-objective multifactorial evolutionary algorithm for assembly line balancing problem considering regular production and preventive maintenance scenarios, *Swarm Evol. Comput.* 68 (2022) 101021, <https://doi.org/10.1016/j.swevo.2021.101021>.
- [57] F. Yin, Z. Du, L. Li, Z. Liang, R. An, R. Wang, G. Liu, Disassembly sequence planning of used smartphone based on dual-population genetic algorithm, *J. Mech. Eng.* 57 (2021) 226–235, <https://doi.org/10.3901/JME.2021.17.226>.
- [58] T.F. Wu, Z.Q. Zhang, T. Yin, Y. Zhang, Multi-objective optimisation for cell-level disassembly of waste power battery modules in human-machine hybrid mode, *Waste Manage.* 144 (2022) 513–526, <https://doi.org/10.1016/j.wasman.2022.04.015>.